LATPC: Accelerating GPU Address Translation Using Locality-Aware TLB Prefetching and MSHR Compression

Yeonan Ha
Yonsei University
Seoul, Republic of Korea
yeonan@yonsei.ac.kr

Jiho Park
Yonsei University
Seoul, Republic of Korea
jihopark@yonsei.ac.kr

Hanna Cha
Yonsei University
Seoul, Republic of Korea
hanna.cha@yonsei.ac.kr

Jiwon Lee
Samsung Electronics
Suwon, Republic of Korea
jiwon24.lee@gmail.com

Joonsung Kim
Sungkyunkwan University
Suwon, Republic of Korea
joonsungkim@skku.edu

Won Woo Ro
Yonsei University
Seoul, Republic of Korea
wro@yonsei.ac.kr

Youngsok Kim*
Yonsei University
Seoul, Republic of Korea
youngsok@yonsei.ac.kr

Abstract

Modern Graphics Processing Units (GPUs) support virtual memory to ease programmability and concurrency, but still suffer from significant address translation overhead due to frequent Translation Lookaside Buffer (TLB) misses and limited TLB Miss-Status Holding Register (MSHR) capacity. These misses trigger long-latency page table walks and stall memory accesses, degrading overall performance. In this paper, we present *LATPC*, a novel mechanism that combines TLB prefetching with MSHR compression to accelerate GPU address translation. LATPC leverages the regularity of Virtual Page Numbers (VPNs) across threads within a warp to coalesce TLB misses at the warp instruction level. LATPC then compresses multiple TLB miss requests into a small number of MSHR entries, reducing contention and enabling more efficient resource use. LATPC further improves translation efficiency by batching page table walks based on the identified VPN patterns, which not only reduces the number of page table walk invocations but also increases off-chip DRAM row buffer locality, lowering the latency of memory accesses during translation. Our evaluation using 24 GPU workloads shows that LATPC effectively exploits regularities and localities in address translation requests within a warp, achieving a 1.47× geometric mean speedup over the baseline without TLB prefetching.

CCS Concepts

• **Computing methodologies** → **Graphics processors**; • **Software and its engineering** → **Virtual memory**.

Keywords

GPU, address translation, TLB prefetching, TLB MSHR compression

ACM Reference Format:
Yeonan Ha, Jiho Park, Hanna Cha, Jiwon Lee, Joonsung Kim, Won Woo Ro, and Youngsok Kim. 2025. LATPC: Accelerating GPU Address Translation Using Locality-Aware TLB Prefetching and MSHR Compression. In *58th IEEE/ACM International Symposium on Microarchitecture (MICRO '25), October 18–22, 2025, Seoul, Republic of Korea.* ACM, New York, NY, USA, 14 pages. https://doi.org/10.1145/3725843.3756069

*Corresponding author

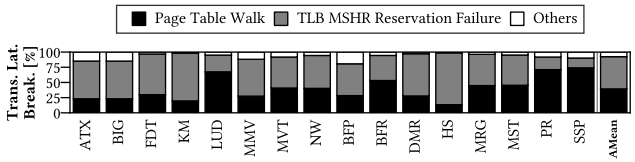


Figure 1: Breakdown of address translation latency. Page table walks and TLB MSHR reservation failures account for a significant portion of address translation latency.

MICRO '25, Seoul, Republic of Korea

ACM ISBN 979-8-4007-1573-0/25/10
https://doi.org/10.1145/3725843.3756069

1 Introduction

Memory virtualization is a crucial feature of modern Graphics Processing Units (GPUs), enhancing programmability, supporting multi-tasking, and providing memory protection [8, 30, 88, 89, 109, 117]. As the off-chip memory of GPUs gets virtualized, accessing off-chip memory (global memory) requires address translation through which Virtual Page Numbers (VPNs) are translated into Physical Frame Numbers (PFNs) [74]. Since a page table, which stores the PFN information, resides in the off-chip memory, address translation involves long-latency page table walks that traverse the page table to find the corresponding PFN for the given VPN [97, 98].

Aimed at reducing the address translation overhead, GPUs employ multi-level and non-inclusive Translation Lookaside Buffers (TLBs) that cache recently translated pairs of VPNs and PFNs [7, 47, 62, 88, 89, 117]. Each GPU core, or Streaming Multiprocessor (SM), has a private TLB (i.e., L1 TLB), while a shared TLB (i.e., L2 TLB) is available at the GPU level, accessible by all SMs. However, the highly limited number of L1 TLB entries (e.g., 16 L1 TLB entries in recent NVIDIA GPUs [117]) makes it difficult to cache all the translations due to the large working sets of GPU workloads [29, 38, 53, 66]. TLB misses occupy the limited number of TLB Miss-Status Holding Register (MSHR) entries (e.g., 12 L1 TLB MSHR entries [91]), which further contribute to address translation overhead [88]. Since the TLB cannot issue more outstanding misses than the number of available MSHR entries [41, 54, 57, 111], insufficient TLB MSHR entries can block subsequent TLB accesses, further exacerbating the overhead of address translation. Moreover, TLB misses incur significant penalties during page table walks due to the limited throughput of page table walkers (e.g., 16 page table

walkers [62]) and the long latency involved in accessing the page table through the L2 cache and off-chip memory (DRAM).

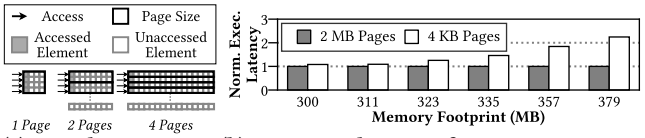
Figure 1 shows the breakdown of address translation latency measured using Accel-Sim [9, 52] across various workloads [22, 26, 27, 40, 103]. The results reveal that two main contributors to address translation latency are page table walks and TLB MSHR reservation failures that account for an average (AMean) of 39.37% and 53.09% of the total address translation latency, respectively. Frequent TLB misses, caused by the limited number of TLB entries, introduce significant overhead to both page table walkers and TLB MSHRs, thereby increasing overall address translation latency. To mitigate address translation overhead, recent studies have explored the use of TLB prefetching mechanisms to reduce TLB miss penalties for both GPUs [14] and CPUs [49, 107, 108]. Unfortunately, existing TLB prefetchers achieve only marginal improvements in TLB miss penalties and performance due to limited prefetching aggressiveness [14] and low prediction accuracy [49, 107, 108].

In this paper, we propose *LATPC*, a novel locality-aware TLB prefetching and TLB MSHR compressing mechanism to accelerate address translation in GPUs by reducing page table walk overheads and mitigating the limitations of insufficient TLB MSHR entries. By exploiting regularities and localities in address translations within a warp memory instruction, LATPC can accurately batch the associated page table walks, and compress the TLB misses into a smaller number of MSHR entries than the number of outstanding TLB misses. First, upon an L1 TLB access, LATPC detects the VPN regularities within the warp memory instruction. This enables the coalescing of multiple translations using the detected regularities. Second, LATPC allocates fewer L1 TLB MSHR entries than the number of missed VPNs within the warp memory instruction. Since LATPC uses a compressed format for allocating an MSHR entry, it conserves L1 TLB MSHR entries, allowing more outstanding TLB misses to be served concurrently and preventing TLB access blocking caused by MSHR exhaustion. Finally, LATPC coalesces multiple page table walks that exhibit locality into a single batched table walk. By exploiting the page table locality, the single batched page table walk accesses only a single DRAM row buffer, resulting in reduced latency when loading the PTEs required by the warp memory instruction.

Our evaluation using Accel-Sim [9, 52] shows LATPC improves performance by achieving a Geometric Mean (GMean) [37] speedup of 1.47× compared to the baseline, which does not employ TLB prefetching [88, 89]. In contrast, existing TLB prefetchers [14, 49, 107, 108] demonstrate only marginal performance improvements due to inaccurate and less aggressive prefetching. They achieve GMean speedups of 1.03×, 0.82×, 1.00×, and 1.01× for Valkyrie, Sequential, Stride, and Distance prefetchers, respectively.

To summarize, this paper makes the following key contributions:

- We observe that the limited numbers of page table walkers and TLB MSHR entries are key bottlenecks in address translation.
- We reveal that access pattern-driven prediction used by existing TLB prefetchers does not work well for GPUs.
- We find that, despite disordered TLB access patterns, the VPNs within a warp instruction exhibit both regularity and locality.
- We propose LATPC, a novel mechanism for locality-aware TLB prefetching and TLB MSHR compression, by exploiting the regularity and locality of translations within a warp instruction.



(a) Page divergence during matrix element loading

(b) Execution latency of GEMM using CUTLASS [76] for different page sizes and memory footprints (normalized to 2 MB pages)

Figure 2: Impact of memory footprint on page divergence and execution latency in matrix multiplication (GEMM)

2 Background

2.1 GPU Address Translation

Graphics Processing Units (GPUs) are equipped with hardware components that provide architectural support for memory virtualization [33, 71, 88, 89]. Each GPU core, or Streaming Multiprocessor (SM), has a private L1 TLB, while all SMs share an L2 TLB. Each L1 TLB includes TLB Miss-Status Holding Registers (MSHRs) [57]. When a warp scheduler issues a global memory instruction, it is sent to the load/store units (LD/ST units) [23, 24], which then probe the L1 TLB for address translation. At the end of LD/ST units, a coalescer coalesces address translation requests of the instruction into the minimum number of requests [88, 89]. The granularity of coalescing is determined by page size (e.g., 4 KB pages [2, 46]). If the probe request misses in the L1 TLB, an L1 TLB MSHR entry is allocated and the request is sent to the L2 TLB; if it also misses in the L2 TLB, the request is sent to a Page Walk Queue (PW Queue) of a GPU Memory Management Unit (GMMU) for page table walks. Page Table Walkers (PTWs) in the GMMU dequeue the request from the PW Queue, walk the page table, and cache recently accessed Page Table Entries (PTEs) in the Page Walk Cache (PW Cache) [11, 17]. After completing page table walks, the retrieved PTE corresponding to the request is inserted into the L2 TLB and propagated to the L1 TLBs. Finally, the associated TLB MSHR entry is released.

2.2 GPU Programming & Page Divergence

GPU programs achieve high parallelism thanks to the large number of available GPU threads. Programmers explicitly map tasks onto GPU threads to fully leverage this capability [28, 39, 70, 102]. Each GPU thread executes its assigned task and accesses corresponding memory based on its thread index [1, 43, 51, 90, 115]. When executing large tasks, increased memory footprints enlarge the dynamic working set of a warp, increasing the number of pages touched by warp memory instructions, thereby requiring multiple translations.

Figure 2 shows the impact of memory footprint on the degree of page divergence and execution latency in matrix multiplication. We define page divergence as the number of unique pages accessed by a warp memory instruction. Figure 2a illustrates page divergence according to matrix size. For a small matrix (small memory footprint), a warp accessing the matrix's rows requires only one address translations, as the access falls within a single page. In the worst case, each thread accesses a different page, meaning that every thread (i.e., 32 threads [74]) requires a unique page. We observe that such page divergence behavior also appears in NVIDIA CUTLASS [76], a vendor-provided library for linear algebra subroutines. Figure 2b shows the execution latency for different page sizes, normalized to

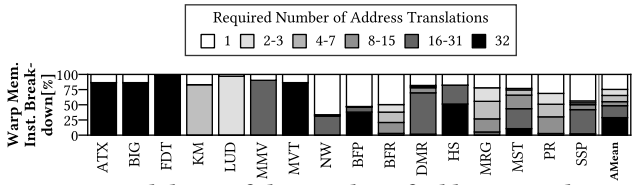


Figure 3: Breakdown of the number of address translations required per warp memory instruction. Warp memory instructions require multiple address translations, which may lead to high contention among TLB MSHRs and PTWs.

the latency of the large page size (i.e., 2 MB pages)[1]. The latency difference between the two page sizes increases with large memory footprints (i.e., large matrix sizes) due to a higher degree of page divergence. These results demonstrate that memory footprint impacts page divergence in common GPU workloads.

Figure 3 shows the breakdown of the number of pages accessed per warp memory instruction. On average, 75.32% of warp instructions exhibit high page divergence (i.e., they require multiple translations). In particular, 28.86% exhibit the maximum page divergence of 32. The results demonstrate that page divergence behavior is common on GPU workloads, and this behavior leads to high TLB misses and high contention among TLB MSHRs and PTWs.

3 Limitations of Prior Approaches

In Figure 1, we demonstrate that page table walks and TLB MSHR reservation failures are the key bottlenecks contributing to address translation latency in GPUs. In this section, we introduce two prior techniques to accelerate address translation: TLB prefetching (Section 3.1) and the use of large page sizes (Section 3.2).

3.1 TLB Prefetching

Prior work has adopted TLB prefetching techniques that prefetch TLB entries ahead of demand accesses aimed at accelerating address translation [14, 19, 49, 107, 108]. Table 1 summarizes the prefetch targets and characteristics of existing TLB prefetchers [14, 49, 107, 108]. The state-of-the-art GPU TLB prefetcher, Valkyrie [14], replicates (prefetches) a PTE into other L1 TLBs that are likely to require it when the PTE is returned from the PTW. However, Valkyrie does not prefetch additional entries from the page table and thus achieves limited performance improvements. We further explore three prominent access-driven prefetchers for CPU TLBs: Sequential [107, 108], Stride [108], and Distance [49, 108] prefetchers. Upon a TLB miss, these prefetchers predict [49, 107, 108] and prefetch additional PTEs that are likely to be used. They predict future PTEs based on observed TLB access patterns and store the prefetched PTEs in an additional on-chip SRAM buffer (i.e., prefetch buffer [108]) located near the L2 TLB. Sequential prefetcher prefetches the next PTE following a missed PTE, while Stride and Distance prefetchers prefetch the k-th subsequent PTE(s) of the missed one, where k is determined based on regularities learned from prior TLB accesses.

Unfortunately, we find that the existing TLB access pattern-driven prefetchers may not be suitable for GPU TLBs, due to their disordered TLB access patterns. Figure 4 shows the performance

[1]We measure the execution latency using NVIDIA Nsight Compute [77] on an NVIDIA RTX 4080 Super GPU [73]. We control the GPU page size by modifying `uvm_va_block_gpu_state_t::force_4k_ptes` variable within the NVIDIA GPU driver [78].

Table 1: Comparison with existing TLB prefetchers

Prefetcher	Prefetch Target	Access Pattern Suitability	
		Strided (Regular)	Non-strided (Irregular)
Valkyrie [14]	Entry replication*	✗	✗
Sequential [107, 108]	+1 entry ahead	✗	✗
Stride [108]	$+s^{\dagger}$ entries ahead	✓	✗
Distance [49, 108]	$+d_1, d_2^{\ddagger}$ entries ahead	✓	✗
LATPC (this work)	Entries required by threads within a warp instruction	✓	✓

* It replicates the entry to L1 TLBs likely to require it, rather than prefetching additional entries.
† s denotes the stride of misses learned per instruction (i.e., per distinct program counter).
‡ d_1 and d_2 denote the distances of misses learned across the TLB.

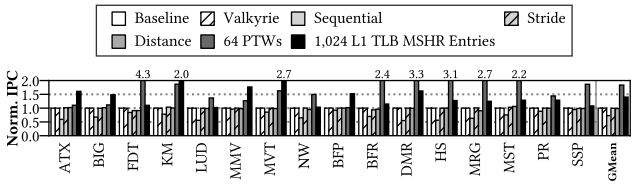


Figure 4: IPC improvements of the existing TLB prefetchers, normalized to the Baseline. The existing prefetchers achieve marginal speedups due to limited prefetching aggressiveness and may degrade performance due to mispredictions.

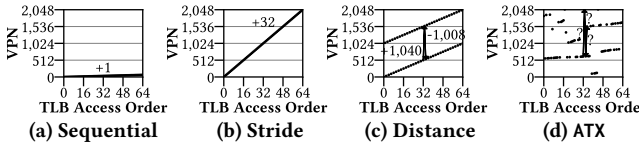


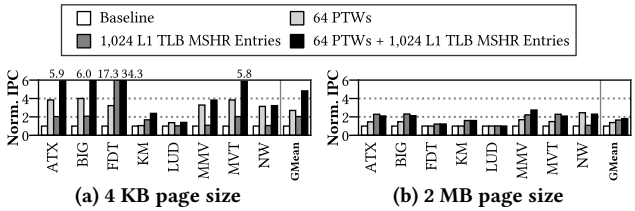
Figure 5: L2 TLB access patterns suitable for (a) Sequential, (b) Stride, and (c) Distance TLB prefetchers, and (d) the L2 TLB access pattern of ATX workload. The existing TLB prefetchers are ineffective for scattered TLB access patterns (e.g., ATX).

improvements of the existing TLB prefetchers. The Baseline does not employ any TLB prefetcher [88, 89]. The existing TLB prefetchers achieve only marginal IPC improvements: 0.73×, 0.99×, 0.98×, and 1.03× for Sequential, Stride, Distance, and Valkyrie prefetchers, respectively. Figure 5 illustrates the L2 TLB access patterns suitable for each of the access pattern-driven prefetchers, and the L2 TLB access patterns of ATX workload. Streaming TLB accesses with a single stride are suitable for Sequential (Figure 5a) and Stride (Figure 5b) prefetchers, while those with multiple strides are suitable for Distance prefetcher (Figure 5c). However, actual GPU workload's L2 TLB access pattern exhibits a scattered pattern (Figure 5d) that makes it difficult to learn any regularity. This suggests that existing access pattern-driven prefetchers may not be suitable for GPUs and highlights the need for an alternative TLB prefetching mechanism.

3.2 Large Page Sizes

Employing large pages (e.g., 2 MB) can alleviate address translation overhead by increasing TLB coverage [15, 32, 58]. Unfortunately, we find that the bottlenecks caused by the limited number of PTWs and L1 TLB MSHR entries still exist with large page sizes. Although employing large page sizes improves TLB coverage, very large memory footprint [29, 38, 53, 66, 97] still results in a high degree of page divergence, similar to that observed with 4 KB pages.

Figure 6 shows the IPC improvements for 4 KB and 2 MB page sizes, which are supported in NVIDIA GPUs [71, 78, 117]. We also show the IPCs achieved when employing a sufficient number of PTWs (i.e., 64 walkers) and L1 TLB MSHR entries (i.e., 1,024 entries),



(a) 4 KB page size (b) 2 MB page size

Figure 6: IPC improvement when resolving PTW and L1 TLB MSHR contentions with (a) 4 KB and (b) 2 MB pages. Regardless of page size, page table walks and L1 TLB MSHR reservation failures remain key bottlenecks in address translation, and resolving them leads to improved performance.

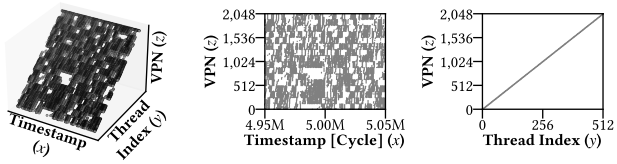
as shown similarly in Figure 4. We select eight workloads whose memory footprints are configurable and enlarge them, following prior work [92]. The memory footprints range from 294.00 MB to 2,296.00 MB (average: 1,141.75 MB) for both the page sizes. For the 4 KB base page size, increasing the number of PTWs and TLB MSHR entries improves IPC by 2.70× and 2.03×, respectively (Figure 6a). For the 2 MB large page size, the corresponding improvements are 1.38× and 1.66× (Figure 6b). Even with larger page sizes, increased memory footprints result in a high degree of page divergence similar to that observed with 4 KB pages, thereby incurring similar contentions. These results demonstrate that the limited number of PTWs and L1 TLB MSHR entries are key bottlenecks, regardless of the page size. Except in Section 6.6, we use a 4 KB page size throughout this paper, following prior works [7, 8, 84, 91].

4 Opportunities

In Section 3.1, we show that the existing TLB prefetchers [14, 49, 107, 108] fail to accelerate address translation, as they cannot capture regularity in the scattered GPU TLB access patterns. However, we observe that there are still opportunities for GPU TLB prefetching by exploiting the regularity and locality of translations within a warp instruction. In this section, we present these opportunities through regularity analysis (Section 4.1) and locality analysis (Section 4.2), for address translations within a warp instruction.

4.1 Regularity Analysis

Figure 7 illustrates the L2 TLB access pattern of ATX from three perspectives: TLB access cycle timestamp (x), index of the requested thread (y), and accessed VPN (z). These visualizations highlight the importance of selecting the appropriate perspective for identifying regular patterns in TLB accesses. The TLB access pattern, represented in a three-dimensional space, forms a plane (Figure 7a). As shown in Figure 5, the existing TLB prefetchers interpret the access pattern with respect to the TLB access cycle timestamp (i.e., xz-plane) (Figure 7b). Whereas no regularity is observed in the projection onto the xz-plane, a clear regular pattern emerges in the projection onto the yz-plane (Figure 7c). The line visible in the yz-plane indicates a strong correlation between the requesting thread indices and the accessed VPNs. Although the TLB access pattern appears highly disordered with respect to the TLB access order, it exhibits strong regularity with respect to the requesting thread index. Therefore, exploiting the regularity with respect to the thread index is well-suited for effective TLB prefetching.



(a) 3D representation of TLB accesses in ATX (b) 2D projection onto the xz-plane (c) 2D projection onto the yz-plane

Figure 7: Visualization of TLB access patterns for ATX. The figures indicate the importance of analyzing VPNs with respect to thread indices, rather than access order (i.e., access cycle timestamps). Here, x denotes the TLB access cycle timestamp, y denotes the thread index, and z denotes the VPN.

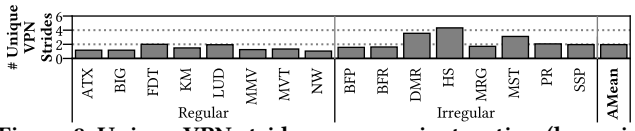


Figure 8: Unique VPN strides per warp instruction (lower is better). Translations within a warp exhibit regularity that can be captured with only a few distinct stride values.

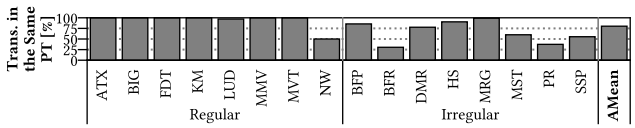


Figure 9: Proportion of address translations in a warp instruction that falls within the same L4 page table (higher is better). Translations within a warp exhibit spatial locality.

Figure 8 shows the number of unique VPN strides within a warp memory instruction, with an average of 1.96 VPN strides. We define a VPN stride as $(\text{VPN}_{i+1} - \text{VPN}_i)$, where VPN_i denotes the required VPN of the i-th thread within a warp instruction. We define the number of unique VPN strides for a warp instruction as $|S|$, where $S = \{(\text{VPN}_{i+1} - \text{VPN}_i) | 0 \leq i < 31, i \in \mathbb{N}\}$. We classify the workloads as either regular or irregular based on whether their memory accesses are determined by thread indices.

The regular workloads have few distinct VPN strides (i.e., 1.42 distinct strides on average), as they access memory in a highly regular pattern. Interestingly, although the irregular workloads do not exhibit an apparent strided pattern in CUDA kernel code, their address translations within the scope of a warp instruction still exhibit regularity (i.e., 2.49 distinct strides on average). The results demonstrate that address translations within a warp memory instruction exhibit an almost-strided pattern, even in the irregular workloads. Therefore, address translations within a warp instruction can be effectively represented and predicted based on this regularity.

4.2 Locality Analysis

Figure 9 shows the proportion of address translations within a warp memory instruction that fall within the same page table level. A high proportion (i.e., 80.20% on average) of translations exhibit spatial locality. For the regular workloads, 93.38% of translations fall within the same L4 PT on average. Similarly, even in irregular workloads, a substantial portion (67.02% on average) of translations exhibit page table locality, as their PTEs reside in the same L4 PT. In this paper, we label the page table levels as L1, L2, L3, and L4 from the root to leaves, following the notation in [67, 93, 96, 105, 110].

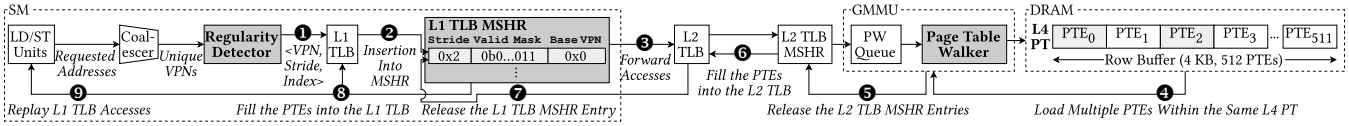
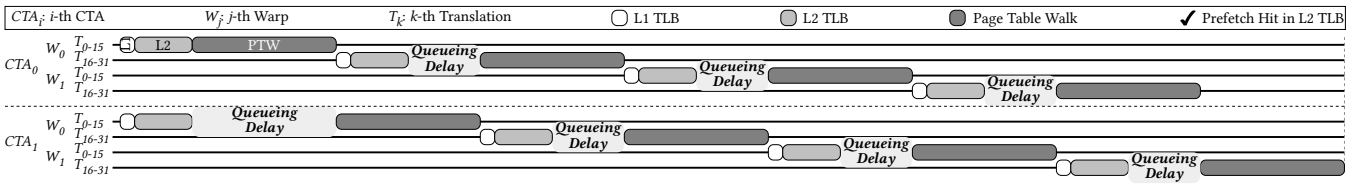
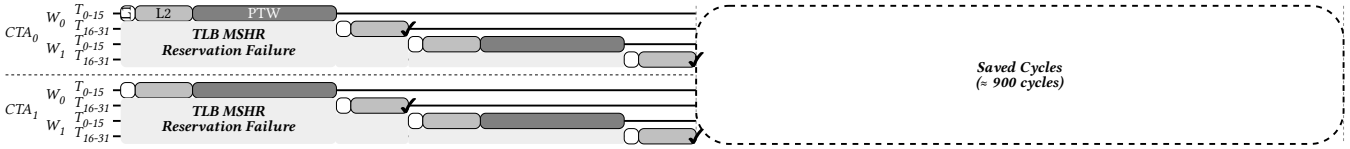


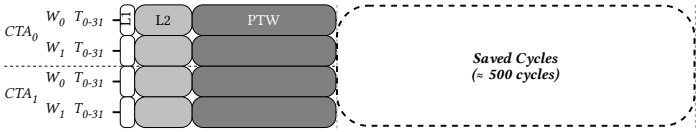
Figure 10: Working model of address translation in LATPC. The added or modified hardware components are shaded.



(a) Baseline. It suffers from queueing delays due to the limited number of PTWs. (~ 1,700 cycles)



(b) Locality-aware TLB prefetching (LATP). It resolves queueing delay by reducing PTW occupancy. (~ 800 cycles)



(c) LATP with locality-aware TLB MSHR compression (LATC) (~ 300 cycles)

Subfigure	Occupancy of		Speedup
	L1 TLB MSHR Entries	PTWs	
(a)	16/16	16/16	1.00×
(b)	16/16	2/16	2.13×
(c)	2/16	4/16	5.67×

(d) Occupancies of TLB MSHR and PTWs, and speedup

Figure 11: Timelines of address translation for the baseline and LATPC (this work)

These results indicate that, irrespective of workloads' memory access characteristic (i.e., regular or irregular), translations within a warp instruction exhibit spatial locality with respect to the page table. When a demand translation request misses in the L1 TLB, the associated prefetch requests within the warp instruction can be handled more efficiently by fetching entries that are likely to reside in the same PT. This observation highlights an opportunity to enable efficient TLB prefetching by leveraging page table locality.

5 LATPC

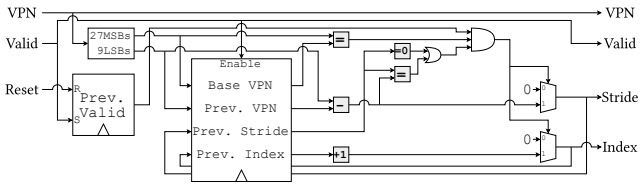
In this section, we propose *LATPC*, a locality-aware TLB prefetching and TLB MSHR compressing mechanism to accelerate GPU address translation. In the remainder of the paper, we refer to the following as: the unit that detects the locality of translations within a warp instruction as Regularity Detector, the mechanism that represents multiple outstanding TLB misses using a single TLB MSHR entry based on locality as Locality-Aware TLB MSHR Compression (LATC), and the mechanism that prefetches multiple PTEs from the PT using locality as Locality-Aware TLB Prefetching (LATP).

5.1 Overview

Figure 10 depicts the working model of LATPC for handling address translations. In the figure, we shade three GPU hardware components that need to be extended to support LATPC: one entirely new unit (i.e., Regularity Detector) and two existing units (i.e., L1 TLB MSHRs and PTWs) that require microarchitectural modifications.

When a warp memory instruction in the LD/ST unit requires address translation, the requests are forwarded to a coalescer, which returns unique VPNs. Regularity Detector processes these VPNs and forwards them to the L1 TLB along with the information <VPN, Stride, Index> (❶). In LATPC, Regularity Detector generates prefetch requests by detecting regularity. We define prefetch translations as translations inferred from detected regularity using Stride and Index. A prefetch request detected by Regularity Detector includes non-zero Stride and non-zero Index values, whereas a demand request has both Stride and Index set to zero. Demand and prefetch translations can be distinguished by whether both Stride and Index are zero or not. Upon an L1 TLB miss, an L1 TLB MSHR entry is allocated, which can represent up to 32 outstanding misses using a compressed format with the given Stride (❷). Subsequent misses that can be represented with the same Base VPN and Stride reuse the existing entry and do not allocate additional ones. The access, along with Stride and Index information, is then forwarded to the L2 TLB. If the access also misses in the L2 TLB, it allocates an L2 TLB MSHR entry, regardless of the request type, and is forwarded to the PW Queue (❸). When a PTW becomes available, it loads PTEs for both the demand translation and the prefetch translations that reside in the same L4 PT, based on the detected locality information (❹). This differs from conventional PTW designs, which handle only one address translation at a time. Similar to Regularity Detector, the LATP-enabled PTW distinguishes prefetch requests by checking whether Stride and Index values are non-zero. Once the PTW completes the page table walk, the allocated L2 TLB MSHR entries are released (❺). After releasing the MSHR entries, the returned PTEs are filled into the L2 TLB (❻). The corresponding L1 TLB MSHR entry is subsequently released (❼) and the loaded PTEs are filled into the L1 TLB (❽). Finally, the L1 TLB MSHR signals the LD/ST unit to replay the address translation that previously missed in the TLB (❾).

Figure 11 illustrates example timelines for handling address translations with LATPC. Each row represents the translation timeline



(a) Hardware of Regularity Detector

Cycle	0	1	2	3	4	5	6	7
Input VPN	0x1000	0x1004	0x1008	0x100c	0x1011	0x1028	0x1025	0x1022
Base VPN	-	0x1000	0x1000	0x1000	0x1000	0x1011	0x1011	0x1025
Prev. VPN	-	0x1000	0x1004	0x1008	0x100c	0x1011	0x1028	0x1025
Prev. Stride	0	0	4	4	4	0	23	0
Prev. Index	0	0	1	2	3	0	1	0
Output	<0x1000, 0, 0>	<0x1004, 4, 1>	<0x1008, 4, 2>	<0x100c, 4, 3>	<0x1011, 0, 0>	<0x1028, 23, 1>	<0x1025, 0, 0>	<0x1022, -3, 1>
Type	Demand	Prefetch	Prefetch	Prefetch	Demand	Prefetch	Demand	Prefetch

(b) Working model of Regularity Detector

Figure 12: (a) Hardware and (b) working model of Regularity Detector. Regularity Detector detects the regularity (i.e., stride) between the previous and current VPNs in each cycle.

for a half warp (the first or last 16 threads in a warp), and the rounded squares denote the latency of TLB accesses or page table walks. In this example, we assume two Cooperative Thread Arrays (CTAs), each consisting of two warps. In the baseline case, subsequent translations experience high queueing delays due to the limited number of PTWs (Figure 11a). After applying LATP, these delays are eliminated as PTW contention is resolved (Figure 11b).

However, L1 TLB MSHR reservation failures still occur, resulting in additional delays. By further applying LATC in addition to LATP, TLB MSHR reservation failures are also completely resolved, allowing all translations to proceed without stalls or delays (Figure 11c). The table in Figure 11d summarizes the occupancy of the 16 L1 TLB MSHR entries and the 16 PTWs for each translation timeline.

5.2 Regularity Detector

Since LATPC highly exploits the stride of VPNs within a warp memory instruction for TLB prefetching, it is important to capture the VPN strides accurately. Figure 12 depicts the microarchitecture and working model of Regularity Detector.

As depicted in Figure 10, Regularity Detector is positioned after the TLB coalescer and takes as input the unique VPNs output by the coalescer. The coalescer identifies and extracts unique VPNs from the virtual addresses required by a warp memory instruction, based on the page size. The order of the output unique VPNs is determined by thread index [52, 79]. Hence, the resulting output can be an unsorted list of VPNs depending on the access pattern, and the stride may be negative. We further discuss the condition under which the unique VPNs are sorted in Section 7.

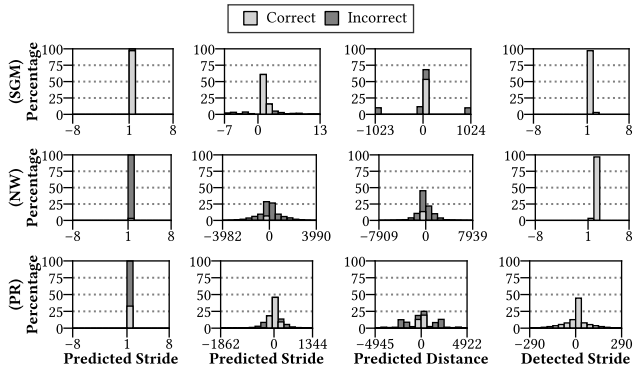
Regularity Detector processes a sequence of unique VPNs, produced by the TLB coalescer for a warp memory instruction, and detects subsequences whose VPNs follow a stride pattern. Each detected subsequence forms a group consisting of one demand request and zero or more prefetch requests. A VPN that does not belong to any stride pattern forms a singleton group consisting of only a demand request. By going through Regularity Detector, each translation request is generated/classified into either demand or prefetch request and transformed into a triple <VPN, Stride, Index>. Demand requests are represented as <VPN, 0, 0> and prefetch requests always have non-zero stride and index, so they can be distinguished by looking at either stride or index. Also, the base VPN of prefetch requests can be calculated from the triples so that the later components can identify the corresponding demand VPN and the group to which the prefetch requests belong. As a result, Regularity Detector transforms a sequence of unique VPNs into a sequence of triples where triples which share the same stride and inferred base VPN form a group of strided VPNs.

Figure 12a illustrates the microarchitecture of Regularity Detector, a state machine composed of a few registers and simple combinational logic. Regularity Detector processes one unique VPN per each cycle and produces a triple <VPN, Stride, Index>. The state registers store 36-bit `Base VPN`, 36-bit `Prev. VPN`, 9-bit `Prev. Stride`, 5-bit `Prev. Index`, and 1-bit `Prev. Valid`. `Prev. Valid` bit is cleared before processing each warp instruction via Reset signal and is set by the first input of the warp instruction. This prevents the state of Regularity Detector from persisting across warp switches. Stride is calculated from the 9 LSBs of the current VPN and `Prev. VPN` using a 9-bit subtractor. We explain the reason for choosing 9 bits in Section 5.4. The MSBs, excluding the 9 LSBs, should match those of `Base VPN`. If either the calculated stride or the MSBs do not match the stored values, the stride is considered discontinued, and the current request becomes a demand request for a new group. The two multiplexers at the end zero out Stride and Index when the current request is a demand. One exception occurs when the `Prev. Stride` is zero, indicating that the previous request was a demand. In this case, stride matching is not required, as depicted in the logic through the OR gate in the figure.

Regularity Detector can detect negative strides by allowing underflows of the subtractor and utilizing potentially underflown Stride values. As a result, the same Stride value may represent either a positive or its negative counterpart. This ambiguity is resolved by computing the base VPN from the triple with only the 9 LSBs, preventing the propagation of the borrow bit.

Figure 12b shows the working model of Regularity Detector, illustrating how register values are updated, and how stride and index information is incorporated into TLB accesses. Regularity Detector starts with `Prev. Valid` register set to invalid. Upon receiving an input VPN `0x1000`, since there is no `Prev. VPN` (i.e., `Prev. Valid` is not set), zero Stride and Index are incorporated into the TLB access, indicating that it is a demand request. At the same time, the register values are updated accordingly. In the next cycle, it receives `0x1004`, and detects the stride of 4. Regularity Detector then incorporates the detected stride and corresponding index into the TLB access, and updates the registers including the `Prev. Stride` and `Prev. Index`. For the next two cycles, Regularity Detector detects the same stride (i.e., 4), so it continues the stride and keeps incrementing the index. At cycle 4, Regularity Detector calculates the new stride to 5, which breaks the stride pattern. Hence, it incorporates zero Stride and Index into the TLB access and resets the `Prev. Stride` and `Prev. Index` registers to zero, terminating the stride sequence. This enables Regularity Detector to detect a new stride of 23 in the following cycle. Since Regularity Detector is capable of detecting a negative stride, it detects the stride of `-3` at cycle 7.

Figure 13 shows the distributions of predicted strides by the existing prefetchers (i.e., Sequential, Stride, and Distance prefetchers) and the detected strides by Regularity Detector. Due to space



(a) Sequential prefetcher (b) Stride prefetcher (c) Distance prefetcher (d) Regularity Detector

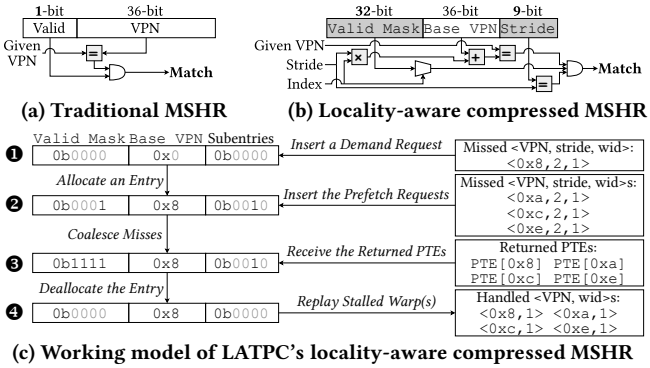
Figure 13: Normalized histograms of predicted strides by existing prefetchers and Regularity Detector for SGM, NW, and PR.

constraints, we present the results from representative workloads in each workload class: SGM, NW, and PR, corresponding to "Regular+Low", "Regular+High", and "Irregular" classes, respectively (see Table 3). For Sequential Prefetcher (Figure 13a), it achieves high accuracy (97.14%) for SGM, but poor accuracy (<35%) for NW and PR. This indicates that a naïve +1 prediction may not always be effective. For Stride and Distance prefetchers (Figures 13b and 13c), although they capture various stride patterns, their accuracy remains below 80% across the workloads. This is because they predict strides or distances based on TLB accesses aggregated from thousands of threads, often leading to inaccurate predictions [35]. Whereas, Regularity Detector not only accurately detects VPN strides within a warp instruction but also detects a wide variety of stride patterns (Figure 13d). Since it detects strides from the coalescer output, which consists of translation requests issued by a warp instruction, it can detect accurate strides regardless of the access pattern. These results demonstrate that Regularity Detector is effective in accurately identifying regularities across diverse workload behaviors.

5.3 L1 TLB MSHR to Enable LATC

To enable LATC, we extend L1 TLB MSHRs by modifying their tag arrays to represent multiple outstanding misses. Figure 14 depicts the comparison logic used in the baseline and LATPC, along with the working model showing how an MSHR entry is managed.

Figure 14a depicts the comparison logic of the existing TLB MSHR tagging. As a traditional MSHR entry can represent only a single outstanding miss [6, 41, 79, 106], the comparison logic includes a single comparator for VPN tagging and a single AND gate for conjunction with the valid bit. Figure 14b depicts the comparison logic required to enable LATC. Whereas traditional MSHR tagging uses a comparator and an AND gate, LATC uses a 9-bit multiplier, an 9-bit adder, two comparators, a 32-to-1 multiplexer (MUX), and an AND gate. We modify the L1 TLB MSHR entry by extending its 1-bit valid bit into a 32-bit `Valid Mask`. This extension allows the compression of up to 32 outstanding TLB misses into a single MSHR entry, ensuring coverage for all the translations within a warp instruction. The i-th bit of `Valid Mask` indicates that the VPN at `Base VPN`+`Stride`×i is in-flight. The subentry does not require modification, as the mechanism for waking up stalled warps



(a) Traditional MSHR (b) Locality-aware compressed MSHR

(c) Working model of LATPC's locality-aware compressed MSHR

Figure 14: (a), (b) Comparison logic for L1 TLB MSHR tagging, and (c) working model of LATPC's compressed MSHR.

Algorithm 1 Operations of LATPC's Compressed TLB MSHR

```
1:  procedure INSERT(VPN, Stride, Index, WarpID)
2:      for each entry E in the MSHR do
3:          if E.Valid = 0x0000_0000 then continue
4:          if ((Stride × Index + E.BaseVPN) = VPN) and (E.Stride = Stride or E.Stride = 0) then
5:              E.Subentry[WarpID] ← 1;  E.Stride ← Stride
6:              if E.Valid[Index] = 1 then return MSHR Hit (Hit-Under-Miss)
7:              E.Valid[Index] ← 1
8:              return MSHR Miss (Miss-Under-Miss)
9:      for each entry E in the MSHR do
10:         if E.Valid = 0x0000_0000 then
11:             E.BaseVPN ← VPN - Stride × Index;  E.Subentry ← 1 << WarpID;  E.Valid[Index] ← 1
12:             return MSHR Miss (Miss-Under-Miss)
13:     return MSHR Reservation Failure
14: procedure ERASE(VPN, Stride, Index)
15:     for each entry E in the MSHR do
16:         if ((Stride × Index + E.BaseVPN) = VPN) and E.Stride = Stride and E.Valid[Index] = 1 then
17:             E.Valid[Index] ← 0
18:             for each warp index j in the SM do
19:                 if E.Subentry[j] = 1 then
20:                     REPLAY(j, VPN)
```

remains the same as that of traditional L1 TLB MSHRs when the missed requests are returned from the L2 TLB.

Figure 14c illustrates an example where the TLB MSHR receives four TLB misses from a warp with an index of 1 (`wid` 1), with VPNs ranging from `0x8` to `0xe` with a stride of 2 (i.e., `0x8`, `0xa`, `0xc`, `0xe`). When the VPN `0x8` misses in the L1 TLB, a new MSHR entry is allocated and the request is forwarded to the L2 TLB. The allocated entry is initialized with the `Base VPN` of `0x8`; the 0-th bit of `Valid Mask` is set, and the first bit of the subentry mask (subentries) is set, indicating that the outstanding miss from warp 1 is being serviced (❶). Since the subsequent misses (i.e., `0xa`, `0xc`, `0xe`) share the same stride as the previous one, they are coalesced into the same entry, and the corresponding bits (i.e., the first, second, and third bits) of `Valid Mask` are set (❷). Each of these misses is then also forwarded to the L2 TLB. After the corresponding PTEs of the four missed VPNs are retrieved from the page table (❸), the responses arrive at the MSHR. The comparison logic uses the index value to reset the corresponding bit of `Valid Mask`. Since the first bit of the subentry mask is set, warp 1 is woken up and replays the L1 TLB access (❹).

Algorithm 1 describes the detailed operations of LATPC's compressed TLB MSHR, including the insertion (allocation) and erasure (release) of entries. `Insert` operation supports acquiring a TLB MSHR entry and returns its status (i.e., MSHR hit, MSHR miss, and MSHR reservation failure). `Erase` operation supports releasing the corresponding L1 TLB MSHR entry once the missed request is returned from a lower level (i.e., L2 TLB), and the L1 TLB accesses are replayed for the missed translations. The comparison logic depicted in Figure 14b is used in Line 4 and Line 16 of `Insert` and `Erase` operations, respectively, for TLB MSHR tag matching.

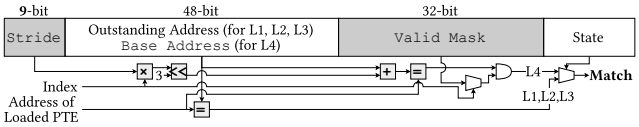


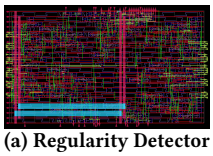
Figure 15: LATP's comparison logic for page walk buffer tagging. A single entry can track multiple PTEs within the same page table level, exploiting the page table locality.

5.4 Page Table Walkers to Enable LATP

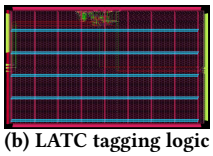
LATP highly exploits the intra-warp VPN stride and DRAM row locality to prefetch the translations within the same warp memory instruction. To enable LATP, we extend the Page Walk Buffers (PW Buffers) [19, 88, 89] so that they can track multiple outstanding walks, in a manner similar to LATC as described in Section 5.3. Translation requests belonging to the same group, as identified by Regularity Detector, are compressed into a single PW Buffer entry using Stride. Since the VPNs in each group are enforced to lie within a 512-page boundary, their corresponding L4 PTEs reside in the same 4 KB DRAM row [48], allowing L4 page table walks to highly exploit DRAM row buffer hits. The boundary constraint also guarantees that the PTEs reside in the same L4 PT, making it possible to complete the common L1-L3 page table walks with a single page table walk per level. This is why we choose 9 bits for strides in Regularity Detector. Therefore, we only extend the page table walk logic for the L4 PT, while keeping the L1-L3 page table walk logic intact. This also allows the physical addresses of outstanding L4 page table walks to be represented with a base address and stride, since the address does not jump into another L4 PT.

When a translation request arrives at the PTW, it first determines whether the request can be coalesced into an existing PW Buffer entry. If not, the PTW allocates a new entry [45, 89] and starts the page table walk by traversing the intermediate levels from the L1 PT down to the L3 PT. During this intermediate-level traversal, subsequent translation requests can be coalesced into the same entry. In such cases, the page table walks for these coalesced translations are reserved until the traversal completes. The partial translation results obtained during the intermediate-level traversal are cached in the Page Walk Cache (PWC) [11, 17]. Once the intermediate-level traversal completes, the PTW issues all the reserved L4 page table walks one by one. Additional translation requests can also be coalesced during the L4 page table walks, and the PTW issues these immediately. The prefetch overhead remains minimal, since all L4 prefetch translations typically hit in the DRAM row buffer. For each completed L4 page table walk, the PTE is filled into the L2 TLB and subsequently forwarded to the corresponding L1 TLBs.

Figure 15 illustrates the extended PW Buffer entry and its tagging logic, similar to LATC, to support multiple outstanding walks for prefetching. In each entry, a 9-bit `Stride` field is added, and the valid bit is extended into a 32-bit `Valid Mask`. For the outstanding address field, LATP uses it as `Base Address` for coalesced translations during L4 page table walks. `Base Address` is always identical to the outstanding address of the demand translation, since the demand request always has an index of zero. To perform tag matching, a 5-bit index value is required, along with the physical address of the loaded PTE. The index is sent to the memory system along with the memory access request and is returned with the fetched data [79]. Using this index, each entry performs stride matching against its `Stride` and `Base Address` to determine whether the walk was initiated by that entry. For L1-L3 page table walks, the entry performs a simple matching operation, identical to that of the baseline PTW, which is selected by the MUX at the end.



(a) Regularity Detector



(b) LATC tagging logic

Process*	Unit	Latency†	Area & Power
45 nm [101]	Regularity Detector	0.3091 ns (1 cycle)	0.0101 mm^2 1.673×10^{-8} mW
	Baseline MSHR	0.2599 ns (1 cycle)	0.0086 mm^2 8.628×10^{-9} mW
	LATC MSHR	0.3054 ns (1 cycle)	0.0087 mm^2 8.934×10^{-9} mW
7 nm [31]	Regularity Detector	0.5047 ns (1 cycle)	0.0009 mm^2 3.924×10^{-9} mW
	Baseline MSHR	0.4648 ns (1 cycle)	0.0007 mm^2 2.8389×10^{-9} mW
	LATC MSHR	0.4986 ns (1 cycle)	0.0008 mm^2 2.9865×10^{-9} mW

* Baseline NVIDIA TU106 GPU's process size: 12 nm [21, 72]
† Baseline NVIDIA TU106 GPU's clock period: 0.73 ns (1,365 MHz) [21, 72]

(c) Hardware implementation cost

Figure 16: Layouts of (a) Regularity Detector and (b) LATC tagging logic, and (c) their hardware implementation cost

5.5 Hardware Implementation Cost

LATPC requires an additional Regularity Detector unit in each SM, and extensions to the L1 TLB MSHR in each SM and the PW Buffer in the GMMU. To estimate the area and power overhead of LATPC, we use OpenROAD flow [4] with two types of process node (i.e., 45 nm [101] and 7 nm [31]) for on-chip logic, and CACTI 7.0 [10] with a 22 nm node for on-chip storage. We also compare the overhead results with the baseline NVIDIA TU106 GPU [21, 72].

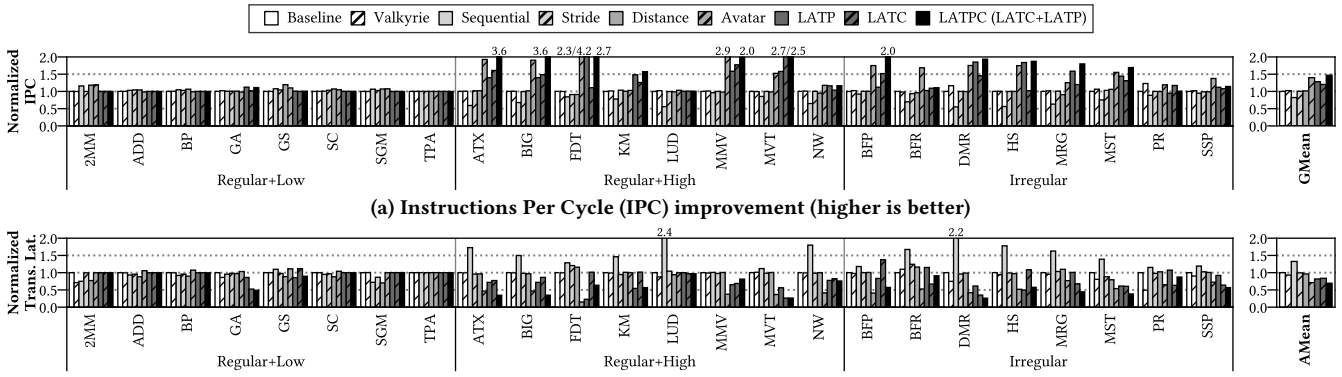
Figure 16 depicts the per-SM layouts of Regularity Detector (Figure 16a) and LATC TLB MSHR tagging logic (Figure 16b) produced by OpenROAD flow [4] using the 7 nm technology ASAP7 library [31]. The table in Figure 16c summarizes the per-GPU implementation costs of Regularity Detector and LATC MSHR tagging logic. Both Regularity Detector and LATC MSHR tagging logic introduce only 1 cycle of additional latency. For the evaluation in Section 6, we model the latency of both components as 1 cycle. The additional latency reduces LATPC's GMean speedup only by 0.45%.

Overall, LATPC introduces 0.2581 mm^2 of additional chip area and 35.78 mW of power consumption, including both on-chip logic and on-chip storage, as well as 2.48 KB of on-chip storage overhead. This corresponds to only 0.05801% of the die size (i.e., 445 mm^2) and 0.02045% of the Thermal Design Power (TDP) (i.e., 175 W) of the baseline NVIDIA TU106 GPU [21, 72]. As discussed in Section 5.3, LATPC only requires extending the 1-bit valid bit into a 32-bit `Valid Mask` and adding a 9-bit `Stride`, and does not incur storage overhead for subentries within an MSHR entry. Each PW Buffer entry requires an additional 9 bits for `Stride` and 32 bits for `Valid Mask`. With 16 L1 TLB MSHR entries per SM across 30 SMs and 16 PW Buffer entries in the GMMU, the total storage requirement of LATPC amounts to 40×16×30 bits = 2,400 bytes for the L1 TLB MSHRs and (9+32)×16 bits = 82 bytes for the PW Buffer.

6 Evaluation

6.1 Experimental Setup

To evaluate LATPC, we use Accel-Sim [9, 52], a cycle-level GPU simulator. We further extend this simulator to accurately and comprehensively model the features of memory virtualization, including



(a) Instructions Per Cycle (IPC) improvement (higher is better)

(b) Address translation latency (lower is better)

Figure 17: (a) Instructions Per Cycle (IPC) improvement and (b) address translation latency of the Baseline, five prior approaches (i.e., Valkyrie [14], Sequential [107, 108], Stride [108], Distance [49, 108], and Avatar [84]), and the three variants of this work (i.e., LATC, LATP, and their combination, LATPC = LATC+LATP). All results are normalized to the Baseline.

Table 2: Simulated system configuration

GPU Configuration	
SM	30 SMs, 4 warp schedulers per SM, 8 warp slots per scheduler Greedy-Then-Oldest (GTO) warp scheduling [95], 1,365 MHz
Cache	128-byte cache line, 32-byte sector [59, 94] (L1 cache) 64 KB, 64-way, 20 cycles (L2 cache) 3 MB, 16-way, 160 cycles
DRAM	GDDR6 [48], 1,750 MHz, 6 GB, 12 channels 16 banks per channel, 336 GB/s, 16 burst length 20-20-20-50 cycles (t_{CL}-t_{RCD}-t_{RP}-t_{RAS}) 4 KB row buffer, 4-4 cycles (t_{CCDS}-t_{CCDL})
Virtual Memory Configuration	
L1 TLB	(4 KB) 32 entries, 16 MSHR entries [57], fully associative (2 MB) 16 entries, 8 MSHR entries, fully associative 20 cycles, 4 ports, Least Recently Used (LRU) replacement policy
L2 TLB	(4 KB) 1,024 entries, 128 MSHR entries, 16-way (2 MB) 128 entries, 128 MSHR entries, 16-way 80 cycles, 16 ports, LRU replacement policy
GMMU	128-entry page walk queue, 16 page table walkers L1/L2/L3 page walk cache [11, 17]: 16/16/16 entries
Page Table	x86-64 4-level page table [2, 46], 4 KB and 2 MB pages

Table 3: List of evaluated workloads

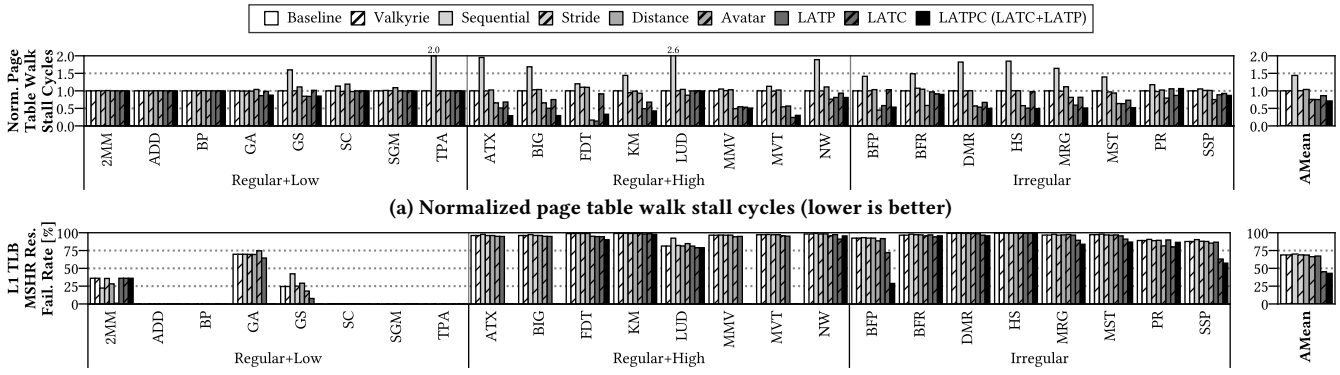
Class	Workload	Abbr.	Memory Access	L2 TLB MPKI		Memory Footprint
Regular+Low	2mm [40]	2MM	Regular	Low	0.64	16.91 MB
	vectorAdd [75]	ADD			3.84	5.73 MB
	backprop [27]	BP			1.43	9.01 MB
	gaussian [27]	GA			4.76	4.70 MB
	gramschmidt [40]	GS			2.44	24.16 MB
	streamcluster [27]	SC			1.39	72.35 MB
	sgemm [103]	SGM			1.73	6.21 MB
	tpacf [103]	TPA			50.00*	0.01 MB
Regular+High	atax [40]	ATX		High	78.46	8.04 MB
	bicg [40]	BIG			78.52	8.04 MB
	fdtd2d [40]	FDT			95.57	6.39 MB
	kmeans [27]	KM			60.97	30.30 MB
	lud [27]	LUD			7.23	102.63 MB
	gesummv [40]	MMV			79.20	6.14 MB
	mvt [40]	MVT			40.00	9.03 MB
	nw [27]	NW			66.26	32.03 MB
Irregular	parboil-bfs [103]	BFP	Irregular		74.16	7.66 MB
	rodinia-bfs [27]	BFR			17.86	37.21 MB
	dmr [22]	DMR			80.20	26.20 MB
	hybridsort [27]	HS			79.35	26.02 MB
	mri-gridding [103]	MRG			22.32	128.82 MB
	mst [22]	MST			15.62	72.99 MB
	pagerank [26]	PR			7.48	19.50 MB
	sssp [22]	SSP			10.63	48.01 MB

* We classify TPA as a low-MPKI workload, as its MPKI is exaggerated. TPA experiences only one L2 TLB miss throughout the end-to-end execution.

multi-level TLBs, the page walk queue, page table walkers, and the Page Walk Cache (PWC) [88, 89], similar to prior work [7, 8, 62, 84, 91]. The average hit rate of PWC for our workloads is 99.99%, 99.93%, and 94.72% for L1 (PML4), L2 (PDP), and L3 (PD) PTs, respectively. Our simulator faithfully models the entire process of address translation, as described in Section 2.1. We use a 4 KB page size for all evaluations, except in Section 6.6, which compares against a 2 MB large page size. Table 2 summarizes the detailed system configuration, which models an NVIDIA RTX 2060-like GPU [21, 72].

We select 24 workloads from CUDA SDK [75], Lonestar [22], Pannotia [26], Parboil [103], Polybench [40], and Rodinia [27] benchmark suites. Similar to prior work [14, 47, 68, 99], we carefully select workloads to stress the virtual memory system. Our selection criteria are high memory footprints exceeding 4 MB (i.e., the coverage of 1,024 L2 TLB entries for a 4 KB page size), and/or high MPKIs exceeding 10 [111]. Our workload suite shares 3 out of 10 workloads with Valkyrie [14] and 8 out of 20 workloads with Avatar [84]. We categorize the workloads into three classes: "Regular+Low", "Regular+High", and "Irregular". "Regular" and "Irregular" classes indicate whether the memory access pattern is regular or irregular. "Regular+Low" and "Regular+High" classes are defined based on their L2 TLB MPKIs, where "Low" denotes MPKI < 5 and "High" denotes MPKI ≥ 5. The memory footprints of the workloads range from 0.01 MB to 128.82 MB, with an average of 29.50 MB. Table 3 lists the evaluated workloads along with their classifications, abbreviations, memory access patterns, L2 TLB MPKIs, and memory footprints.

We compare LATPC with four existing TLB prefetchers introduced in Table 1: Valkyrie [14], Sequential [107, 108], Stride [108], and Distance [49, 108] prefetchers. We also compare LATPC with a recent approach, Avatar [84], the state-of-the-art in GPU address translation. We implement each of the prior works as described in their original papers to the best of our knowledge [14, 84, 108]. We implement Valkyrie to prefetch the returned PTE into the L1 TLBs, based on decisions made using its Locality Detection Table (LDT), and to probe the other L1 TLBs upon an L1 TLB miss. LDT is placed near the L2 TLB and configured with 100 entries, and the L1 TLBs are connected with a bidirectional ring network [14]. For Sequential, Stride, and Distance prefetchers, we implement them by placing the prefetchers near the L2 TLB to prefetch predicted PTEs from the page table. Both the prediction table and the prefetch buffer are located near the L2 TLB, and their capacities are set to 64 entries [108]. We implement Avatar to perform speculative address translation using its Mapping Offset Detection (MOD) table when a translation request misses in the L1 TLB. MOD table is placed near each L1 TLB, configured with 32 entries [84]. For LATPC, we model the additional latency of Regularity Detector and LATC MSHR tagging logic as 1 cycle, as discussed in Section 5.5.



(a) Normalized page table walk stall cycles (lower is better)

(b) L1 TLB MSHR reservation failure rate (lower is better). No reservation failures are observed for `ADD`, `BP`, `SC`, `SGM`, and `TPA` workloads.

Figure 18: (a) Page table walk stall cycles, normalized to the Baseline, and (b) L1 TLB MSHR reservation failure rate

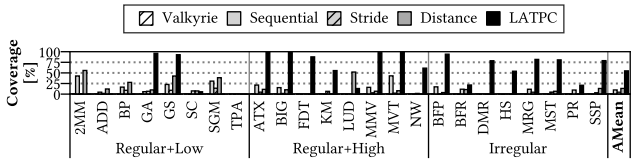
6.2 Performance Analysis

We evaluate the IPC improvement of LATPC against the Baseline [88, 89] and compare the results with five prior approaches: Valkyrie [14], Sequential [107, 108], Stride [108], Distance [49, 108] prefetchers, and Avatar [84]. To calculate the average results across the workloads, we use the Geometric Mean (GMean) [37] for IPC improvement and the Arithmetic Mean (AMean) for other metrics.

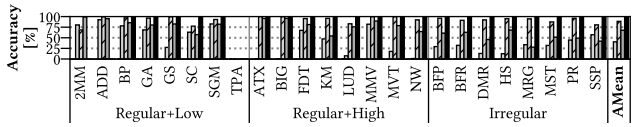
Figure 17 shows the IPC improvements and address translation latencies, normalized to the Baseline. LATPC achieves a GMean speedup of 1.47×, while the existing approaches achieve 1.03×, 0.82×, 1.00×, 1.01×, and 1.40× for Valkyrie, Sequential, Stride, Distance, and Avatar, respectively (Figure 17a). These results demonstrate that LATPC outperforms all the existing approaches, including Avatar, the state-of-the-art GPU address translation technique. This improvement comes from LATPC's ability to effectively accelerate address translation by alleviating the contentions on both PTWs and L1 TLB MSHRs. Valkyrie, the state-of-the-art GPU TLB prefetcher, delivers only a small performance improvement, since it only replicates the entry into the L1 TLBs when an L2 TLB miss is handled, rather than prefetching additional entries from the page table. Furthermore, this replication is sufficiently performed by the L2 TLB MSHR in the Baseline. Stride and Distance prefetchers achieve minor performance improvements due to their relatively inaccurate prefetching, and Sequential prefetcher degrades performance as it naïvely prefetches the next entry for each miss, thus further exacerbating contention on PTWs and L1 TLB MSHRs.

We further present the IPC improvements achieved by applying each idea of LATPC (i.e., LATP and LATC). Applying LATP and LATC results in GMean speedups of 1.28× and 1.20×, respectively. These results indicate that both ideas of LATPC are equally important for accelerating GPU address translation. LATP is effective in resolving queueing delays caused by the limited number of PTWs, while LATC helps mitigate L1 TLB MSHR reservation failures due to the limited number of L1 TLB MSHR entries.

LATPC reduces the normalized address translation latency to 69.46%, while Valkyrie, Stride, Distance, and Avatar reduce it to 93.10%, 99.68%, 96.27%, and 71.24%, respectively (Figure 17b). The aggressive yet inaccurate prefetching of Sequential prefetcher instead increases it (i.e., 132.58%). These results demonstrate that LATPC significantly accelerates address translation, thanks to its ability to alleviate contention on PTWs and L1 TLB MSHRs.



(a) Coverage (higher is better)



(b) Accuracy (higher is better)

Figure 19: (a) Prefetch coverage and (b) accuracy of the four existing TLB prefetchers and LATPC (this work)

6.3 Hardware Contention Analysis

The decreased address translation latency, shown in Section 6.2, is due to alleviated contention in translation hardware (i.e., PTWs and L1 TLB MSHRs). We demonstrate the effectiveness of LATP in reducing page table walk stall cycles, defined as queueing delays caused by limited PTW availability in the PW Queue [91], and of LATC in lowering L1 TLB MSHR reservation failure rates.

Figure 18 shows the page table walk stall cycles, normalized to the Baseline, and the L1 TLB MSHR reservation failure rates. LATPC reduces the normalized page table walk stall cycles to 71.20%, while Avatar, the state-of-the-art GPU address translation technique, reduces it to 75.54% (Figure 18a). They similarly mitigate the PTW contention through different mechanisms: LATPC does so by exploiting the page table locality of translation requests within a warp instruction, whereas Avatar reduces the contention by speculating physical addresses without performing address translation. However, Avatar cannot alleviate the L1 TLB MSHR contention, as it still has to acquire an MSHR entry during speculation. The L1 TLB MSHR reservation failure rates are reduced to 43.02% and 66.33% for LATPC and Avatar, respectively, from the Baseline of 68.84% (Figure 18b). The other approaches (i.e., Valkyrie, Sequential, Stride, and Distance prefetchers) cannot effectively reduces page table walk stall cycles and L1 TLB MSHR reservation failures.

6.4 Prefetch Coverage and Accuracy Analysis

We compare the prefetch coverage and accuracy of LATPC against the existing TLB prefetchers [14, 49, 107, 108]. We define prefetch

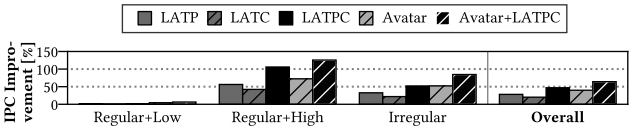


Figure 20: Performance comparison between Avatar and Avatar+LATPC across different workload classes. Avatar shows improved performance when integrated with LATPC.

coverage as the fraction of TLB misses that become TLB hits, and prefetch accuracy as the fraction of correct TLB prefetches among all TLB prefetches, following the definitions introduced in [34, 100].

Figure 19 shows the prefetch coverage and accuracy of the existing TLB prefetchers and LATPC. LATPC effectively eliminates TLB misses by achieving a high prefetch coverage of 54.78%, while the existing TLB prefetchers achieve less than 15% on average (Figure 19a). Prefetch coverage is a first-order metric that is critical to performance, as it represents the removal of TLB misses. LATPC achieves significant performance improvements as a result of its high prefetch coverage. Not only does LATPC achieve high prefetch coverage, but it also achieves high prefetch accuracy because LATP exactly prefetches the required translations based on the regularity information provided by Regularity Detector (Figure 19b). For `2MM`, `ADD`, and `TPA`, LATPC does not perform any TLB prefetching due to their low page divergence. Since it does not issue any incorrect prefetching requests, its prefetching accuracy is actually 100%. Nonetheless, to avoid overstating the benefits, we report the prefetching accuracy as 0% for these three workloads. The results demonstrate that LATPC outperforms existing TLB prefetchers.

6.5 Comparison with Avatar

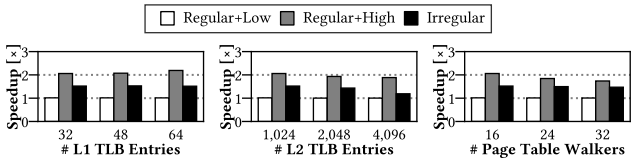
We evaluate the performance of Avatar [84], the state-of-the-art in GPU address translation, when integrated with LATPC. Avatar and LATPC are orthogonal, as Avatar focuses on speculative address translation upon an L1 TLB miss, while LATPC mitigates the contentions on L1 TLB MSHRs and PTWs caused by the L1 TLB miss.

Figure 20 shows the GMean speedup of three proposals of this study (i.e., LATP, LATC, and LATPC), Avatar, and Avatar+LATPC. Avatar achieves a lower GMean speedup of 1.40× compared to LATPC (i.e., 1.47×), as it still suffers from high contention in translation components (e.g., 66.33% of L1 TLB MSHR reservation failure rate). However, Avatar+LATPC achieves a GMean speedup of 1.64× and reduces the L1 TLB MSHR reservation failure rate to 39.73%. Although Avatar performs speculative translation upon an L1 TLB miss, the actual address translation is still performed in the background [84]. Both TLB MSHRs and PTWs are occupied until the speculation is validated. Furthermore, when a cache sector is incompressible, Avatar cannot benefit from speculation, and execution must stall until background translation is completed. Avatar+LATPC mitigates contention in TLB MSHRs and PTWs during speculation, and accelerates background translation.

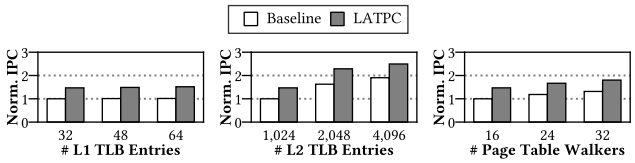
6.6 Comparison to Large Page Sizes

Employing large pages [15, 32, 58] can mitigate address translation overhead by increasing TLB reach. However, even with large page sizes, a huge memory footprint still causes a high degree of page divergence, leading to high contentions on PTWs and TLB MSHRs.

Figure 21: IPC improvement when employing 2 MB pages and LATPC. LATPC achieves superior performance even with large page sizes. IPC results are normalized to 2 MB pages.



(a) LATPC's speedup over the baseline for different workload classes



(b) Normalized IPC of the baseline and LATPC (normalized to the baseline's smallest configuration in each component)

Figure 22: Sensitivity study of L1/L2 TLB entries and PTWs. (a) LATPC consistently achieves high speedup across different configurations and workload classes. (b) LATPC often outperforms the baseline, even with smaller configurations.

LATPC effectively mitigates these contentions not only with 4 KB base pages but also with 2 MB large pages [71, 78, 117]. We evaluate LATPC with 2 MB large pages for all the workloads listed in Table 3. As discussed in Section 3.2, the memory footprints of workloads in "Regular+High" class are enlarged, following prior work [92]. The average memory footprint of "Regular+High" class is 1.14 GB, and we do not enlarge the footprints for other workload classes.

Figure 21 shows the IPC improvements of LATPC with a 2 MB page size, compared to the baseline using the same page size. The impact of LATPC remains significant even when adopting a large page size, achieving a GMean speedup of 1.18×. LATPC continues to mitigate contention on PTWs and TLB MSHRs and improve performance due to the high degree of page divergence, even with 2 MB large pages. The results demonstrate LATPC's effectiveness in accelerating address translation, regardless of the page size.

6.7 Sensitivity Study

To examine the ability of LATPC under larger TLB and PTW configurations, we conduct a sensitivity study on the number of L1 TLB entries, L2 TLB entries, and PTWs. For the 32-entry L1 TLB and 16-walker PTW configurations, we present results for our configurations scaled by 1.0×, 1.5×, and 2.0×. In particular, for the L2 TLB, we show results for 1,024, 2,048, and 4,096 entries, to align with the 4,096-entry L2 TLB configuration of NVIDIA Ampere GPUs [117].

Figure 22 presents the results of our sensitivity study for varying configurations. Although LATPC's speedup slightly decreases as the configuration size increases, it still achieves high speedup

across all workload classes (Figure 22a). We further report the IPCs, normalized to the IPC of the baseline's smallest configuration for each component (Figure 22b). Interestingly, LATPC with 2,048 L2 TLB entries outperforms the baseline with 4,096 L2 TLB entries, and LATPC with 16 PTWs outperforms the baseline with 32 PTWs. For an Ampere GPU with 4,096 L2 TLB entries, employing LATPC provides higher performance while saving (4,096 - 2,048) × 8-byte = 16 KB of on-chip storage. These results imply that LATPC achieves high speedup regardless of the hardware configuration and can even outperform the baseline with lower hardware cost.

7 Discussions

Sorting Requirement of Regularity Detector. Regularity Detector operates irrespective of whether the unique VPNs are sorted, as it detects regularity between the previous and current VPNs in a cycle-by-cycle manner. When the VPNs are sorted, LATPC achieves an additional GMean speedup of 3% across our workloads compared to the unsorted case, as sorting enables Regularity Detector to detect more regularities among VPNs. However, sorting introduces an additional latency of $\sum_{i=1}^{\log_2 32} \log_2 2^i = \sum_{i=1}^{5} i = 15$ stages/cycles when using a bitonic sorter [16]. Due to its high additional latency and chip area cost (i.e., 454.39× greater than that of our Regularity Detector), our evaluation does not consider VPN sorting.

Valkyrie's Effectiveness. Despite of our faithful implementation, the performance gap between our evaluation and the original paper [14] primarily stems from the L2 TLB MSHR subentry configuration and the low inter-TLB locality in our memory footprints. When adjusting the L2 TLB MSHR configuration (i.e., reducing the number of L2 TLB MSHR subentries to 1), Valkyrie's GMean speedup across our 24 workloads increases from 1.03× to 1.20×. Since the indices of missed SM are stored in the L2 TLB MSHR subentries [84], it plays a role similar to Valkyrie's prefetching mechanism by passing the returned PTE from the PTW to the missed L1 TLBs. We further analyze inter-TLB locality across the three workloads that used in both our evaluation and the original paper. For SGM, KM, and PR, pages are shared by 29.31, 1.49, and 2.15 SMs on average, with respective speedups of 1.00×, 1.01×, and 1.23× under our large footprint. The relatively small speedup of SGM stems from its limited potential (i.e., 1.07× when using an always-hit TLB), despite exhibiting high locality. Meanwhile, KM's small speedup is attributed to its low locality (i.e., 1.49 among 30 SMs) under our memory footprint. On the other hand, PR achieves a significant speedup thanks to the increase in L1 TLB hit rate (i.e., increasing from 19.99% to 59.67%) facilitated by Valkyrie's probing mechanism. For MM from AMD APP SDK benchmark suite [104] and KM, Valkyrie achieves speedups of 1.53× and 2.15×, respectively, with memory footprints of high inter-TLB locality (i.e., page sharing of 26.88 and 14.42 SMs). Similar to the recent papers [35, 36] that observe less than 1.01× speedup in Valkyrie, we focus on large memory footprints to focus on the problems caused by page divergence.

8 Related Work

8.1 Accelerating GPU Address Translation

There has been extensive research on address translation in GPUs, similar to that in CPUs [3, 18, 20, 25, 42, 44, 50, 68, 80–83, 85, 86, 99, 105, 114]. Lowe-Power et al. [89] and Pichai et al. [88] explored hardware support for address translation in GPUs. Vesely et al. [109] investigated a virtual memory system for integrated GPUs. Ausavarungnirun et al. [7] proposed page promotion by transforming base pages into page groups. Lee et al. [62] extended page promotion by recursively merging pages or page groups into larger page groups. Shin et al. [97, 98] and Lee et al. [60] enhanced page table walkers by leveraging the locality of page tables and the page table walk characteristics of applications. Recently, Park et al. [84] proposed a speculative translation [5, 12, 87] approach in GPUs.

There have also been attempts to leverage underutilized resources in the last-level cache [47], on-chip components such as the instruction cache and scratchpad memory [56], and the Ray Tracing (RT) core [35]. Various environments and configurations of GPUs have been considered, including virtual cache hierarchies [116], multi-tenant GPUs [8, 91], multi-GPU setups [13, 63–65, 112, 113], and multi-chip-module (MCM) GPUs [36, 92]. These studies are orthogonal to the problem addressed in this paper.

8.2 GPU Data Prefetching

Some studies have investigated methods to mitigate the overhead of long-latency data cache misses [55, 61, 69]. Lee et al. [61] introduced the first GPU data cache prefetcher. Koo et al. [55] extended the scope of prefetching from the warp to the CTA. Mostofi et al. [69] captured not only fixed strides but also variable strides by utilizing a chain of strides. In contrast to data cache prefetching, Ganguly et al. [38] studied page prefetching and page eviction mechanisms for memory oversubscription environments. Although prior work has attempted to mitigate the overhead of data cache misses and page migration, none of these studies has addressed GPU TLB prefetching to reduce address translation overhead.

9 Conclusion

We proposed LATPC, a novel mechanism for locality-aware TLB prefetching and TLB MSHR compression. We observed that warp instructions often demanded multiple address translations, and their translations exhibited both regularity and locality. To exploit these opportunities, first, we introduced Regularity Detector, which detected the locality of VPNs within a warp instruction. Second, we extended the L1 TLB MSHRs with LATC, allowing each MSHR entry to represent multiple TLB-missed VPNs by compressing them based on their VPN locality. Third, we extended PTWs with LATP, enabling a PTW to load multiple PTEs by batching page table walks to the page table level. LATPC significantly reduced address translation overhead by resolving the bottlenecks caused by limited PTWs and L1 TLB MSHRs. LATPC achieved a 1.47× geometric mean speedup over the baseline, without employing TLB prefetching.

Acknowledgments

This work was partly supported by the National Research Foundation of Korea (NRF) grants (RS-2025-00513906, RS-2025-00521413) and the Institute of Information & communications Technology Planning & Evaluation (IITP) grants (RS-2020-II201361, RS-2024-00395134, RS-2025-02217106, RS-2025-02304554) funded by the Korea government (MSIT).

References

[1] Tor M Aamodt, Wilson Wai Lun Fung, Timothy G Rogers, and Margaret Martonosi. 2018. *General-Purpose Graphics Processor Architectures*. Morgan & Claypool Publishers. 12–13 pages.

[2] Advanced Micro Devices, Inc. 2024. AMD64 Architecture Programmer's Manual. https://www.amd.com/content/dam/amd/en/documents/processor-tech-docs/programmer-references/40332.pdf.

[3] Jeongseob Ahn, Seongwook Jin, and Jaehyuk Huh. 2012. Revisiting Hardware-Assisted Page Walks for Virtualized Systems. In *Proc. ISCA*.

[4] Tutu Ajayi, Vidya A Chhabria, Mateus Fogaça, Soheil Hashemi, Abdelrahman Hosny, Andrew B Kahng, Minsoo Kim, Jeongsup Lee, Uday Mallappa, Marina Neseem, et al. 2019. Toward an Open-Source Digital Flow: First Learnings from the OpenROAD Project. In *Proc. DAC*.

[5] Chloe Alverti, Stratos Psomadakis, Vasileios Karakostas, Jayneel Gandhi, Konstantinos Nikas, Georgios Goumas, and Nectarios Koziris. 2020. Enhancing and Exploiting Contiguity for Fast Memory Virtualization. In *Proc. ISCA*.

[6] Mikhail Asiatici and Paolo Ienne. 2019. Stop Crying Over Your Cache Miss Rate: Handling Efficiently Thousands of Outstanding Misses in FPGAs. In *Proc. FPGA*.

[7] Rachata Ausavarungnirun, Joshua Landgraf, Vance Miller, Saugata Ghose, Jayneel Gandhi, Christopher J. Rossbach, and Onur Mutlu. 2017. Mosaic: A GPU Memory Manager with Application-Transparent Support for Multiple Page Sizes. In *Proc. MICRO*.

[8] Rachata Ausavarungnirun, Vance Miller, Joshua Landgraf, Saugata Ghose, Jayneel Gandhi, Adwait Jog, Christopher J. Rossbach, and Onur Mutlu. 2018. MASK: Redesigning the GPU Memory Hierarchy to Support Multi-Application Concurrency. In *Proc. ASPLOS*.

[9] Ali Bakhoda, George L Yuan, Wilson WL Fung, Henry Wong, and Tor M Aamodt. 2009. Analyzing CUDA Workloads Using a Detailed GPU Simulator. In *Proc. ISPASS*.

[10] Rajeev Balasubramonian, Andrew B Kahng, Naveen Muralimanohar, Ali Shafiee, and Vaishnav Srinivas. 2017. CACTI 7: New Tools for Interconnect Exploration in Innovative Off-Chip Memories. *ACM TACO* 14, 2 (2017).

[11] Thomas W Barr, Alan L Cox, and Scott Rixner. 2010. Translation Caching: Skip, Don't Walk (the Page Table). In *Proc. ISCA*.

[12] Thomas W. Barr, Alan L. Cox, and Scott Rixner. 2011. SpecTLB: A Mechanism for Speculative Address Translation. In *Proc. ISCA*.

[13] Trinayan Baruah, Yifan Sun, Ali Tolga Dinçer, Saiful A. Mojumder, José L. Abellán, Yash Ukidave, Ajay Joshi, Norman Rubin, John Kim, and David Kaeli. 2020. Griffin: Hardware-Software Support for Efficient Page Migration in Multi-GPU Systems. In *Proc. HPCA*.

[14] Trinayan Baruah, Yifan Sun, Saiful A Mojumder, José L Abellán, Yash Ukidave, Ajay Joshi, Norman Rubin, John Kim, and David Kaeli. 2020. Valkyrie: Leveraging Inter-TLB Locality to Enhance GPU Performance. In *Proc. PACT*.

[15] Arkaprava Basu, Jayneel Gandhi, Jichuan Chang, Mark D. Hill, and Michael M. Swift. 2013. Efficient Virtual Memory for Big Memory Servers. In *Proc. ISCA*.

[16] Kenneth E Batcher. 1968. Sorting networks and their applications. In *Proc. AFIPS*.

[17] Abhishek Bhattacharjee. 2013. Large-Reach Memory Management Unit Caches. In *Proc. MICRO*.

[18] Abhishek Bhattacharjee. 2017. Translation-Triggered Prefetching. In *Proc. ASPLOS*.

[19] Abhishek Bhattacharjee. 2019. Appendix L: Advanced Concepts on Address Translation. https://www.cs.yale.edu/homes/abhishek/abhishek-appendix-l.pdf.

[20] Abhishek Bhattacharjee and Margaret Martonosi. 2010. Inter-Core Cooperative TLB Prefetchers for Chip Multiprocessors. In *Proc. ASPLOS*.

[21] John Burgess. 2020. RTX on—The NVIDIA Turing GPU. *IEEE Micro* 40, 2 (2020).

[22] Martin Burtscher, Rupesh Nasre, and Keshav Pingali. 2012. A Quantitative Study of Irregular Programs on GPUs. In *Proc. IISWC*.

[23] Hanna Cha, Sungchul Lee, Yeonan Ha, Hanhwi Jang, Joonsung Kim, and Youngsok Kim. 2024. GCStack: A GPU Cycle Accounting Mechanism for Providing Accurate Insight into GPU Performance. *IEEE CAL* 23, 2 (2024).

[24] Hanna Cha, Sungchul Lee, Jounghoo Lee, Yeonan Ha, Joonsung Kim, and Youngsok Kim. 2025. GCStack+GCScaler: Fast and Accurate GPU Performance Analyses Using Fine-Grained Stall Cycle Accounting and Interval Analysis. In *Proc. ISCA*.

[25] Dimitrios Chasapis, Georgios Vavouliotis, Daniel A. Jiménez, and Marc Casas. 2025. Instruction-Aware Cooperative TLB and Cache Replacement Policies. In *Proc. ASPLOS*.

[26] Shuai Che, Bradford M Beckmann, Steven K Reinhardt, and Kevin Skadron. 2013. Pannotia: Understanding Irregular GPGPU Graph Applications. In *Proc. IISWC*.

[27] Shuai Che, Michael Boyer, Jiayuan Meng, David Tarjan, Jeremy W. Sheaffer, Sang-Ha Lee, and Kevin Skadron. 2009. Rodinia: A Benchmark Suite for Heterogeneous Computing. In *Proc. IISWC*.

[28] Shuai Che, Michael Boyer, Jiayuan Meng, David Tarjan, Jeremy W Sheaffer, and Kevin Skadron. 2008. A performance study of general-purpose applications on graphics processors using CUDA. *JPDC* 68, 10 (2008).

[29] Sangjin Choi, Taeksoo Kim, Jinwoo Jeong, Rachata Ausavarungnirun, Myeongjae Jeon, Youngjin Kwon, and Jeongseob Ahn. 2022. Memory Harvesting in Multi-GPU Systems with Hierarchical Unified Virtual Memory. In *Proc. ATC*.

[30] Jack Choquette. 2023. NVIDIA Hopper H100 GPU: Scaling Performance. *IEEE Micro* 43, 3 (2023).

[31] Lawrence T Clark, Vinay Vashishtha, Lucian Shifren, Aditya Gujja, Saurabh Sinha, Brian Cline, Chandarasekaran Ramamurthy, and Greg Yeric. 2016. ASAP7: A 7-nm finFET predictive process design kit. *Microelectronics* 53 (2016).

[32] Guilherme Cox and Abhishek Bhattacharjee. 2017. Efficient Address Translation for Architectures with Multiple Page Sizes. In *Proc. ASPLOS*.

[33] Radoslav Danilak. 2009. Systems and Methods for Hardware-Based GPU Paging to System. US Patent No. 7,623,134, Nov. 24th, 2009.

[34] Babak Falsafi and Thomas F. Wenisch. 2014. *A Primer on Hardware Prefetching*. Morgan & Claypool Publishers, Chapter 1, 3–4.

[35] Yuan Feng, Yuke Li, Jiwon Lee, Won Woo Ro, and Hyeran Jeon. 2025. Heliostat: Harnessing Ray Tracing Accelerators for Page Table Walks. In *Proc. ISCA*.

[36] Yuan Feng, Seonjin Na, Hyesoon Kim, and Hyeran Jeon. 2024. Barre Chord: Efficient Virtual Memory Translation for Multi-Chip-Module GPUs. In *Proc. ISCA*.

[37] Philip J. Fleming and John J. Wallace. 1986. How Not to Lie With Statistics: The Correct Way to Summarize Benchmark Results. *CACM* 29, 3 (1986).

[38] Debashis Ganguly, Ziyu Zhang, Jun Yang, and Rami Melhem. 2019. Interplay between Hardware Prefetcher and Page Eviction Policy in CPU-GPU Unified Virtual Memory. In *Proc. ISCA*.

[39] Michael Garland, Scott Le Grand, John Nickolls, Joshua Anderson, Jim Hardwick, Scott Morton, Everett Phillips, Yao Zhang, and Vasily Volkov. 2008. Parallel Computing Experiences with CUDA.

[40] Scott Grauer-Gray, Lifan Xu, Robert Searles, Sudhee Ayalasomayajula, and John Cavazos. 2012. Auto-tuning a High-Level Language Targeted to GPU Codes. In *Proc. InPar*.

[41] Yongbin Gu and Lizhong Chen. 2019. Dynamically linked MSHRs for adaptive miss handling in GPUs. In *Proc. ICS*.

[42] Faruk Guvenilir and Yale N Patt. 2020. Tailored Page Sizes. In *Proc. ISCA*.

[43] Dongho Ha, Yunho Oh, and Won Woo Ro. 2023. R2D2: Removing ReDunDancy Utilizing Linearity of Address Generation in GPUs. In *Proc. ISCA*.

[44] Swapnil Haria, Mark D. Hill, and Michael M. Swift. 2018. Devirtualizing Memory in Heterogeneous Systems. In *Proc. ASPLOS*.

[45] Gur Hildesheim, Chang Kian Tan, Robert S Chappell, and Rohit Bhatia. 2015. Concurrent Page Table Walker Control for TLB Miss Handling. US Patent No. 9,069,690, Jun. 30th., 2015.

[46] Intel Corporation. 2024. Intel® 64 and IA-32 Architectures Software Developer's Manual. https://cdrdv2-public.intel.com/835781/325462-sdm-vol-1-2abcd-3abcd-4.pdf.

[47] Aamer Jaleel, Eiman Ebrahimi, and Sam Duncan. 2019. DUCATI: High-Performance Address Translation by Extending TLB Reach of GPU-Accelerated Systems. *ACM TACO* 16 (2019).

[48] JEDEC Solid State Technology Association. 2021. Graphics Double Data Rate (GDDR6) SGRAM Standard. JESD250D. https://www.jedec.org/system/files/docs/JESD250D.pdf.

[49] Gokul B Kandiraju and Anand Sivasubramaniam. 2002. Going the Distance for TLB Prefetching: An Application-driven Study. In *Proc. ISCA*.

[50] Vasileios Karakostas, Jayneel Gandhi, Furkan Ayar, Adrián Cristal, Mark D Hill, Kathryn S McKinley, Mario Nemirovsky, Michael M Swift, and Osman Ünsal. 2015. Redundant Memory Mappings for Fast Access to Large Memories. In *Proc. ISCA*.

[51] Mahmoud Khairy, Vadim Nikiforov, David Nellans, and Timothy G. Rogers. 2020. Locality-Centric Data and Threadblock Management for Massive GPUs. In *Proc. MICRO*.

[52] Mahmoud Khairy, Zhesheng Shen, Tor M. Aamodt, and Timothy G. Rogers. 2020. Accel-Sim: An Extensible Simulation Framework for Validated GPU Modeling. In *Proc. ISCA*.

[53] Hyojong Kim, Jaewoong Sim, Prasun Gera, Ramyad Hadidi, and Hyesoon Kim. 2020. Batch-Aware Unified Memory Management in GPUs for Irregular Workloads. In *Proc. ASPLOS*.

[54] Youngsok Kim, Jaewon Lee, Jae-Eon Jo, and Jangwoo Kim. 2014. GPUdmm: A High-Performance and Memory-Oblivious GPU Architecture Using Dynamic Memory Management. In *Proc. HPCA*.

[55] Gunjae Koo, Hyeran Jeon, Zhenhong Liu, Nam Sung Kim, and Murali Annavaram. 2018. CTA-Aware Prefetching and Scheduling for GPU. In *Proc. IPDPS*.

[56] Jagadish B. Kotra, Michael LeBeane, Mahmut T. Kandemir, and Gabriel H. Loh. 2021. Increasing GPU Translation Reach by Leveraging Under-Utilized On-Chip Resources. In *Proc. MICRO*.

[57] David Kroft. 1981. Lockup-Free Instruction Fetch/Prefetch Cache Organization. In *Proc. ISCA*.

[58] Youngjin Kwon, Hangchen Yu, Simon Peter, Christopher J Rossbach, and Emmett Witchel. 2016. Coordinated and Efficient Huge Page Management with Ingens. In *Proc. OSDI*.

[59] Jounghoo Lee, Yeonan Ha, Suhyun Lee, Jinyoung Woo, Jinho Lee, Hanhwi Jang, and Youngsok Kim. 2022. GCoM: A Detailed GPU Core Model for Accurate Analytical Modeling of Modern GPUs. In *Proc. ISCA*.

[60] Jiwon Lee, Gun Ko, Myung Kuk Yoon, Ipoom Jeong, Yunho Oh, and Won Woo Ro. 2025. Marching Page Walks: Batching and Concurrent Page Table Walks for Enhancing GPU Throughput. In *Proc. HPCA*.

[61] Jaekyu Lee, Nagesh B. Lakshminarayana, Hyesoon Kim, and Richard Vuduc. 2010. Many-Thread Aware Prefetching Mechanisms for GPGPU Applications. In *Proc. MICRO*.

[62] Jiwon Lee, Ju Min Lee, Yunho Oh, William J Song, and Won Woo Ro. 2023. SnakeByte: A TLB Design with Adaptive and Recursive Page Merging in GPUs. In *Proc. HPCA*.

[63] Bingyao Li, Yueqi Wang, Tianyu Wang, Lieven Eeckhout, Jun Yang, Aamer Jaleel, and Xulong Tang. 2024. STAR: Sub-Entry Sharing-Aware TLB for Multi-Instance GPU. In *Proc. MICRO*.

[64] Bingyao Li, Jieming Yin, Anup Holey, Youtao Zhang, Jun Yang, and Xulong Tang. 2023. Trans-FW: Short Circuiting Page Table Walk in Multi-GPU Systems via Remote Forwarding. In *Proc. HPCA*.

[65] Bingyao Li, Jieming Yin, Youtao Zhang, and Xulong Tang. 2021. Improving Address Translation in Multi-GPUs via Sharing and Spilling Aware TLB Design. In *Proc. MICRO*.

[66] Chen Li, Rachata Ausavarungnirun, Christopher J Rossbach, Youtao Zhang, Onur Mutlu, Yang Guo, and Jun Yang. 2019. A Framework for Memory Oversubscription Management in Graphics Processing Units. In *Proc. ASPLOS*.

[67] Artemiy Margaritov, Dmitrii Ustiugov, Edouard Bugnion, and Boris Grot. 2018. Virtual Address Translation via Learned Page Table Indexes. In *Workshop on ML for Systems at NeurIPS*.

[68] Artemiy Margaritov, Dmitrii Ustiugov, Edouard Bugnion, and Boris Grot. 2019. Prefetched Address Translation. In *Proc. MICRO*.

[69] Saba Mostofi, Hajar Falahati, Negin Mahani, Pejman Lotfi-Kamran, and Hamid Sarbazi-Azad. 2023. Snake: A Variable-length Chain-based Prefetching for GPUs. In *Proc. MICRO*.

[70] Aaftab Munshi. 2009. The OpenCL Specification. In *Proc. HCS*.

[71] NVIDIA Corporation. 2016. Pascal MMU Format Change. https://nvidia.github.io/open-gpu-doc/pascal/gp100-mmu-format.pdf.

[72] NVIDIA Corporation. 2018. NVIDIA Turing GPU Architecture. https://www.nvidia.com/content/dam/en-zz/Solutions/design-visualization/technologies/turing-architecture/NVIDIA-Turing-Architecture-Whitepaper.pdf.

[73] NVIDIA Corporation. 2023. NVIDIA Ada GPU Architecture. https://images.nvidia.com/aem-dam/Solutions/geforce/ada/nvidia-ada-gpu-architecture.pdf.

[74] NVIDIA Corporation. 2025. CUDA C++ Programming Guide. https://docs.nvidia.com/cuda/pdf/CUDA_C_Programming_Guide.pdf.

[75] NVIDIA Corporation. 2025. CUDA Samples. https://github.com/nvidia/cuda-samples.

[76] NVIDIA Corporation. 2025. CUTLASS. https://github.com/nvidia/cutlass.

[77] NVIDIA Corporation. 2025. Nsight Compute Documentation. https://docs.nvidia.com/nsight-compute.

[78] NVIDIA Corporation. 2025. NVIDIA Linux Open GPU Kernel Module Source. https://github.com/NVIDIA/open-gpu-kernel-modules.

[79] Lars Nyland, John R. Nickolls, Gentaro Hirota, and Tanmoy Mandal. 2011. Systems and Methods for Coalescing Memory Accesses of Parallel Threads. US Patent No. 8,086,806, Dec. 27th., 2011.

[80] Chang Hyun Park, Sanghoon Cha, Bokyeong Kim, Youngjin Kwon, David Black-Schaffer, and Jaehyuk Huh. 2020. Perforated Page: Supporting Fragmented Memory Allocation for Large Pages. In *Proc. ISCA*.

[81] Chang Hyun Park, Taekyung Heo, and Jaehyuk Huh. 2016. Efficient Synonym Filtering and Scalable Delayed Translation for Hybrid Virtual Caching. In *Proc. ISCA*.

[82] Chang Hyun Park, Taekyung Heo, Jungi Jeong, and Jaehyuk Huh. 2017. Hybrid TLB Coalescing: Improving TLB Translation Coverage under Diverse Fragmented Memory Allocations. In *Proc. ISCA*.

[83] Chang Hyun Park, Ilias Vougioukas, Andreas Sandberg, and David Black-Schaffer. 2022. Every Walk's a Hit: Making Page Walks Single-Access Cache Hits. In *Proc. ASPLOS*.

[84] Junhyeok Park, Kwon Osang, Yongho Lee, Seongwook Kim, Gwangeun Byeon, Jihun Yoon, Prashant J Nair, and Seokin Hong. 2024. A Case for Speculative Address Translation with Rapid Validation for GPUs. In *Proc. MICRO*.

[85] Binh Pham, Abhishek Bhattacharjee, Yasuko Eckert, and Gabriel H. Loh. 2014. Increasing TLB Reach by Exploiting Clustering in Page Translations. In *Proc. HPCA*.

[86] Binh Pham, Viswanathan Vaidyanathan, Aamer Jaleel, and Abhishek Bhattacharjee. 2012. CoLT: Coalesced Large-Reach TLBs. In *Proc. MICRO*.

[87] Binh Pham, Ján Veselỳ, Gabriel H Loh, and Abhishek Bhattacharjee. 2015. Large Pages and Lightweight Memory Management in Virtualized Environments: Can You Have it Both Ways?. In *Proc. MICRO*.

[88] Bharath Pichai, Lisa Hsu, and Abhishek Bhattacharjee. 2014. Architectural Support for Address Translation on GPUs: Designing Memory Management Units for CPU/GPUs with Unified Address Spaces. In *Proc. ASPLOS*.

[89] Jason Power, Mark D. Hill, and David A. Wood. 2014. Supporting x86-64 Address Translation for 100s of GPU lanes. In *Proc. HPCA*.

[90] B Pratheek, Guilherme Cox, Jan Vesely, and Arkaprava Basu. 2024. SUV: Static analysis guided Unified Virtual Memory. In *Proc. MICRO*.

[91] B. Pratheek, Neha Jawalkar, and Arkaprava Basu. 2021. Improving GPU Multi-tenancy with Page Walk Stealing. In *Proc. HPCA*.

[92] B Pratheek, Neha Jawalkar, and Arkaprava Basu. 2022. Designing Virtual Memory System of MCM GPUs. In *Proc. MICRO*.

[93] Antonis Psistakis, Nikos Chrysos, Fabien Chaix, Marios Asiminakis, Michalis Gianioudis, Pantelis Xirouchakis, Vassilis Papaefstathiou, and Manolis Katevenis. 2022. Optimized Page Fault Handling During RDMA. *TPDS* 33, 12 (2022).

[94] Minsoo Rhu, Michael Sullivan, Jingwen Leng, and Mattan Erez. 2013. A Locality-Aware Memory Hierarchy for Energy-Efficient GPU Architectures. In *Proc. MICRO*.

[95] Timothy G Rogers, Mike O'Connor, and Tor M Aamodt. 2012. Cache-Conscious Wavefront Scheduling. In *Proc. MICRO*.

[96] Robert Schilling, Pascal Nasahl, Stefan Weiglhofer, and Stefan Mangard. 2021. SecWalk: Protecting Page Table Walks Against Fault Attacks. In *Proc. HOST*.

[97] Seunghee Shin, Guilherme Cox, Mark Oskin, Gabriel H. Loh, Yan Solihin, Abhishek Bhattacharjee, and Arkaprava Basu. 2018. Scheduling Page Table Walks for Irregular GPU Applications. In *Proc. ISCA*.

[98] Seunghee Shin, Michael LeBeane, Yan Solihin, and Arkaprava Basu. 2018. Neighborhood-Aware Address Translation for Irregular GPU Applications. In *Proc. MICRO*.

[99] Dimitrios Skarlatos, Apostolos Kokolis, Tianyin Xu, and Josep Torrellas. 2020. Elastic Cuckoo Page Tables: Rethinking Virtual Memory Translation for Parallelism. In *Proc. ASPLOS*.

[100] Yan Solihin. 2015. *Fundamentals of Parallel Multicore Architecture*. Chapman & Hall/CRC, Chapter 5, 162–163.

[101] James E Stine, Ivan Castellanos, Michael Wood, Jeff Henson, Fred Love, W Rhett Davis, Paul D Franzon, Michael Bucher, Sunil Basavarajaiah, Julie Oh, et al. 2007. FreePDK: An Open-Source Variation-Aware Design Kit. In *Proc. MSE*.

[102] John E. Stone, David Gohara, and Guochun Shi. 2010. OpenCL: A Parallel Programming Standard for Heterogeneous Computing Systems. *IEEE CiSE* 12, 3 (2010).

[103] John A Stratton, Christopher Rodrigues, I-Jui Sung, Nady Obeid, Li-Wen Chang, Nasser Anssari, Geng Daniel Liu, and Wen-mei W Hwu. 2012. Parboil: A Revised Benchmark Suite for Scientific and Commercial Throughput Computing. *IMPACT Technical Report* (2012).

[104] Yifan Sun, Trinayan Baruah, Saiful A. Mojumder, Shi Dong, Xiang Gong, Shane Treadway, Yuhui Bao, Spencer Hance, Carter McCardwell, Vincent Zhao, Harrison Barclay, Amir Kavyan Ziabari, Zhongliang Chen, Rafael Ubal, José L. Abellán, John Kim, Ajay Joshi, and David Kaeli. 2019. MGPUSim: Enabling Multi-GPU Performance Modeling and Optimization. In *Proc. ISCA*.

[105] Madhusudhan Talluri, Mark D Hill, and Yousef A Khalidi. 1995. A New Page Table for 64-bit Address Spaces. In *Proc. SOSP*.

[106] Blaise Tine, Krishna Praveen Yalamarthy, Fares Elsabbagh, and Kim Hyesoon. 2021. Vortex: Extending the RISC-V ISA for GPGPU and 3D-Graphics. In *Proc. MICRO*.

[107] Steven P Vanderwiel and David J Lilja. 2000. Data Prefetch Mechanisms. *ACM CSUR* 32, 2 (2000).

[108] Georgios Vavouliotis, Lluc Alvarez, Vasileios Karakostas, Konstantinos Nikas, Nectarios Koziris, Daniel A Jiménez, and Marc Casas. 2021. Exploiting Page Table Locality for Agile TLB Prefetching. In *Proc. ISCA*.

[109] Jan Vesely, Arkaprava Basu, Mark Oskin, Gabriel H Loh, and Abhishek Bhattacharjee. 2016. Observations and Opportunities in Architecting Shared Virtual Memory for Heterogeneous Systems. In *Proc. ISPASS*.

[110] Ilias Vougioukas. 2022. How about a short walk? https://community.arm.com/arm-research/b/articles/posts/how-about-a-short-walk.

[111] Lu Wang, Magnus Jahre, Almutaz Adileh, and Lieven Eeckhout. 2020. MDM: The GPU Memory Divergence Model. In *Proc. MICRO*.

[112] Yueqi Wang, Bingyao Li, Aamer Jaleel, Jun Yang, and Xulong Tang. 2024. GRIT: Enhancing Multi-GPU Performance with Fine-Grained Dynamic Page Placement. In *Proc. HPCA*.

[113] Yueqi Wang, Bingyao Li, Mohamed Tarek Ibn Ziad, Lieven Eeckhout, Jun Yang, Aamer Jaleel, and Xulong Tang. 2025. OASIS: Object-Aware Page Management for Multi-GPU Systems. In *Proc. HPCA*.

[114] Zi Yan, David Nellans, Daniel Lustig, and Abhishek Bhattacharjee. 2019. Translation Ranger: Operating System Support for Contiguity-Aware TLBs. In *Proc. ISCA*.

[115] Tsung Tai Yeh, Roland N Green, and Timothy G Rogers. 2020. Dimensionality-Aware Redundant SIMT Instruction Elimination. In *Proc. ASPLOS*.

[116] Hongil Yoon, Jason Lowe-Power, and Gurindar S Sohi. 2018. Filtering Translation Bandwidth with Virtual Caching. In *Proc. ASPLOS*.

[117] Zhenkai Zhang, Tyler Allen, Fan Yao, Xing Gao, and Rong Ge. 2023. TunneLs for Bootlegging: Fully Reverse-Engineering GPU TLBs for Challenging Isolation Guarantees of NVIDIA MIG. In *Proc. CCS*.